Eigensystems

CHAPTER 11

11.0 Introduction

An $N \times N$ matrix $\mathbf{A}$ is said to have an *eigenvector* $\mathbf{x}$ and corresponding *eigenvalue* λ if

$$\mathbf{A} \cdot \mathbf{x} = \lambda \mathbf{x} \tag{11.0.1}$$

Obviously any multiple of an eigenvector $\mathbf{x}$ will also be an eigenvector, but we won't consider such multiples as being distinct eigenvectors. (The zero vector is not considered to be an eigenvector at all.) Evidently (11.0.1) can hold only if

$$\det |\mathbf{A} - \lambda \mathbf{1}| = 0 \tag{11.0.2}$$

which, if expanded out, is an Nth degree polynomial in λ whose roots are the eigenvalues. This proves that there are always N (not necessarily distinct) eigenvalues. Equal eigenvalues coming from multiple roots are called *degenerate*. Root searching in the characteristic equation (11.0.2) is usually a very poor computational method for finding eigenvalues. We will learn much better ways in this chapter, as well as efficient ways for finding corresponding eigenvectors.

The above two equations also prove that every one of the N eigenvalues has a (not necessarily distinct) corresponding eigenvector: If λ is set to an eigenvalue, then the matrix $\mathbf{A} - \lambda \mathbf{1}$ is singular, and we know that every singular matrix has at least one nonzero vector in its nullspace (see §2.6.1).

If you add $\tau \mathbf{x}$ to both sides of (11.0.1), you will easily see that the eigenvalues of any matrix can be changed or *shifted* by an additive constant τ by adding to the matrix that constant times the identity matrix. The eigenvectors are unchanged by this shift. Shifting, as we will see, is an important part of many algorithms for computing eigenvalues. We see also that there is no special significance to a zero eigenvalue. Any eigenvalue can be shifted to zero, or any zero eigenvalue can be shifted away from zero.

11.0.1 Definitions and Basic Facts

A matrix is called *symmetric* if it is equal to its transpose,

$$\mathbf{A} = \mathbf{A}^T \qquad \text{or} \qquad a_{ij} = a_{ji} \tag{11.0.3}$$

It is called *Hermitian* or *self-adjoint* if it equals the complex conjugate of its transpose (its *Hermitian conjugate*, denoted by "†")

$$\mathbf{A} = \mathbf{A}^\dagger \qquad \text{or} \qquad a_{ij} = a_{ji}* \tag{11.0.4}$$

It is termed *orthogonal* if its transpose equals its inverse,

$$\mathbf{A}^T \cdot \mathbf{A} = \mathbf{A} \cdot \mathbf{A}^T = \mathbf{1} \tag{11.0.5}$$

and *unitary* if its Hermitian conjugate equals its inverse. Finally, a matrix is called *normal* if it *commutes* with its Hermitian conjugate,

$$\mathbf{A} \cdot \mathbf{A}^\dagger = \mathbf{A}^\dagger \cdot \mathbf{A} \tag{11.0.6}$$

For real matrices, Hermitian means the same as symmetric, unitary means the same as orthogonal, and *both* of these distinct classes are normal.

The reason that Hermitian is an important concept has to do with eigenvalues. The eigenvalues of a Hermitian matrix are all real. In particular, the eigenvalues of a real symmetric matrix are all real. Contrariwise, the eigenvalues of a real nonsymmetric matrix may include real values, but may also include pairs of complex-conjugate values; and the eigenvalues of a complex matrix that is not Hermitian will in general be complex.

The reason that normal is an important concept has to do with the eigenvectors. The eigenvectors of a normal matrix with nondegenerate (i.e., distinct) eigenvalues are complete and orthogonal, spanning the N-dimensional vector space. For a normal matrix with degenerate eigenvalues, we have the additional freedom of replacing the eigenvectors corresponding to a degenerate eigenvalue by linear combinations of themselves. Using this freedom, we can always perform Gram-Schmidt orthogonalization (consult any linear algebra text) and *find* a set of eigenvectors that are complete and orthogonal, just as in the nondegenerate case. The matrix whose columns are an orthonormal set of eigenvectors is evidently unitary. A special case is that the matrix of eigenvectors of a real symmetric matrix is orthogonal, since the eigenvectors of that matrix are all real.

When a matrix is not normal, as typified by any random, nonsymmetric, real matrix, then in general we cannot find *any* orthonormal set of eigenvectors, nor even any pairs of eigenvectors that are orthogonal (except perhaps by rare chance). While the N nonorthonormal eigenvectors will "usually" span the N-dimensional vector space, they do not always do so; that is, the eigenvectors are not always complete. Such a matrix is said to be *defective*.

11.0.2 Left and Right Eigenvectors

While the eigenvectors of a nonnormal matrix are not particularly orthogonal among themselves, they *do* have an orthogonality relation with a different set of vectors, which we must now define. Up to now our eigenvectors have been column vectors that are multiplied to the right of a matrix $\mathbf{A}$, as in (11.0.1). These, more explicitly, are termed *right eigenvectors*. We could also, however, try to find row vectors, which multiply $\mathbf{A}$ to the left and satisfy

$$\mathbf{x} \cdot \mathbf{A} = \lambda \mathbf{x} \tag{11.0.7}$$

These are called *left eigenvectors*. By taking the transpose of equation (11.0.7), we see that every left eigenvector is the transpose of a right eigenvector *of the transpose of* $\mathbf{A}$. Now, by comparing to (11.0.2) and using the fact that the determinant of a matrix equals the determinant of its transpose, we also see that the left and right eigen*values* of $\mathbf{A}$ are identical.

If the matrix $\mathbf{A}$ is symmetric, then the left and right eigenvectors are just transposes of each other, that is, they have the same numerical components. Likewise, if the matrix is self-adjoint, the left and right eigenvectors are Hermitian conjugates of each other. For the general nonnormal case, however, we have the following calculation: Let $\mathbf{X}_R$ be the matrix formed by columns from the right eigenvectors and $\mathbf{X}_L$ be the matrix formed by rows from the left eigenvectors. Then (11.0.1) and (11.0.7) can be rewritten as

$$\mathbf{A} \cdot \mathbf{X}_R = \mathbf{X}_R \cdot \mathrm{diag}(\lambda_0 \ldots \lambda_{N-1}) \qquad \mathbf{X}_L \cdot \mathbf{A} = \mathrm{diag}(\lambda_0 \ldots \lambda_{N-1}) \cdot \mathbf{X}_L \quad (11.0.8)$$

Multiplying the first of these equations on the left by $\mathbf{X}_L$, the second on the right by $\mathbf{X}_R$, and subtracting the two, gives

$$(\mathbf{X}_L \cdot \mathbf{X}_R) \cdot \mathrm{diag}(\lambda_0 \ldots \lambda_{N-1}) = \mathrm{diag}(\lambda_0 \ldots \lambda_{N-1}) \cdot (\mathbf{X}_L \cdot \mathbf{X}_R) \qquad (11.0.9)$$

This says that the matrix of dot products of the left and right eigenvectors commutes with the diagonal matrix of eigenvalues. But the only matrices that commute with a diagonal matrix *of distinct elements* are themselves diagonal. Thus, if the eigenvalues are nondegenerate, each left eigenvector is orthogonal to all right eigenvectors except its corresponding one, and vice versa. By choice of normalization, the dot products of corresponding left and right eigenvectors can always be made unity for any matrix with nondegenerate eigenvalues.

If some eigenvalues are degenerate, then either the left or the right eigenvectors corresponding to a degenerate eigenvalue must be linearly combined among themselves to achieve orthogonality with the right or left ones, respectively. This can always be done by a procedure akin to Gram-Schmidt orthogonalization. The normalization can then be adjusted to give unity for the nonzero dot products between corresponding left and right eigenvectors. If the dot product of corresponding left and right eigenvectors is zero at this stage, then you have a case where the eigenvectors are incomplete! Note that incomplete eigenvectors can occur only where there are degenerate eigenvalues, but they do not always occur in such cases (in fact, they never occur for the class of "normal" matrices). See [1] for a clear discussion.

In both the degenerate and nondegenerate cases, the final normalization to unity of all nonzero dot products produces the result: The matrix whose rows are left eigenvectors is the inverse matrix of the matrix whose columns are right eigenvectors, *if the inverse exists*. When it does exist, equations (11.0.8) and (11.0.9) imply the useful factorizations

$$\mathbf{A} = \mathbf{X}_R \cdot \mathrm{diag}(\lambda_0 \ldots \lambda_{N-1}) \cdot \mathbf{X}_L \quad \text{and} \quad \mathrm{diag}(\lambda_0 \ldots \lambda_{N-1}) = \mathbf{X}_L \cdot \mathbf{A} \cdot \mathbf{X}_R \quad (11.0.10)$$

11.0.3 Diagonalization of a Matrix

From equation (11.0.10) and the fact that $\mathbf{X}_L$ and $\mathbf{X}_R$ are matrix inverses, we get

$$\mathbf{X}_R^{-1} \cdot \mathbf{A} \cdot \mathbf{X}_R = \mathrm{diag}(\lambda_0 \ldots \lambda_{N-1}) \qquad (11.0.11)$$

This is a particular case of a *similarity transform* of the matrix $\mathbf{A}$,

$$\mathbf{A} \quad \rightarrow \quad \mathbf{Z}^{-1} \cdot \mathbf{A} \cdot \mathbf{Z} \tag{11.0.12}$$

for some transformation matrix $\mathbf{Z}$. Similarity transformations play a crucial role in the computation of eigenvalues, because they leave the eigenvalues of a matrix unchanged. This is easily seen from

$$\begin{aligned} \det\left|\mathbf{Z}^{-1} \cdot \mathbf{A} \cdot \mathbf{Z} - \lambda \mathbf{1}\right| &= \det\left|\mathbf{Z}^{-1} \cdot (\mathbf{A} - \lambda \mathbf{1}) \cdot \mathbf{Z}\right| \\ &= \det|\mathbf{Z}|\ \det|\mathbf{A} - \lambda \mathbf{1}|\ \det\left|\mathbf{Z}^{-1}\right| \\ &= \det|\mathbf{A} - \lambda \mathbf{1}| \end{aligned} \tag{11.0.13}$$

Equation (11.0.11) shows that any matrix with complete eigenvectors (which includes all normal matrices and "most" random nonnormal ones) can be diagonalized by a similarity transformation, that the columns of the transformation matrix that effects the diagonalization are the right eigenvectors, and that the rows of its inverse are the left eigenvectors.

For real symmetric matrices, the eigenvectors are real and orthonormal, so the transformation matrix is orthogonal. The similarity transformation is then also an *orthogonal transformation* of the form

$$\mathbf{A} \quad \rightarrow \quad \mathbf{Z}^{T} \cdot \mathbf{A} \cdot \mathbf{Z} \tag{11.0.14}$$

While real nonsymmetric matrices can be diagonalized in their usual case of complete eigenvectors, the transformation matrix is not necessarily real. It turns out, however, that a real similarity transformation can "almost" do the job. It can reduce the matrix down to a form with little two-by-two blocks along the diagonal and all other elements zero. Each two-by-two block corresponds to a complex-conjugate pair of complex eigenvalues. We will see this idea exploited in some routines given later in the chapter.

The "grand strategy" of virtually all modern eigensystem routines is to nudge the matrix $\mathbf{A}$ toward diagonal form by a sequence of similarity transformations,

$$\begin{aligned} \mathbf{A} \quad &\rightarrow \quad \mathbf{P}_1^{-1} \cdot \mathbf{A} \cdot \mathbf{P}_1 \quad \rightarrow \quad \mathbf{P}_2^{-1} \cdot \mathbf{P}_1^{-1} \cdot \mathbf{A} \cdot \mathbf{P}_1 \cdot \mathbf{P}_2 \\ &\rightarrow \quad \mathbf{P}_3^{-1} \cdot \mathbf{P}_2^{-1} \cdot \mathbf{P}_1^{-1} \cdot \mathbf{A} \cdot \mathbf{P}_1 \cdot \mathbf{P}_2 \cdot \mathbf{P}_3 \quad \rightarrow \quad \text{etc.} \end{aligned} \tag{11.0.15}$$

If we get all the way to diagonal form, then the eigenvectors are the columns of the accumulated transformation

$$\mathbf{X}_R = \mathbf{P}_1 \cdot \mathbf{P}_2 \cdot \mathbf{P}_3 \cdot \ldots \tag{11.0.16}$$

Sometimes we do not want to go all the way to diagonal form. For example, if we are interested only in eigenvalues, not eigenvectors, it is enough to transform the matrix $\mathbf{A}$ to be triangular, with all elements below (or above) the diagonal zero. In this case the diagonal elements are already the eigenvalues, as you can see by mentally evaluating (11.0.2) using expansion by minors.

There are two rather different sets of techniques for implementing the grand strategy (11.0.15). It turns out that they work rather well in combination, so most modern eigensystem routines use both. The first set of techniques constructs individual $\mathbf{P}_i$'s as explicit "atomic" transformations designed to perform specific tasks,

for example zeroing a particular off-diagonal element (Jacobi transformation, §11.1), or a whole particular row or column (Householder transformation, §11.3; elimination method, §11.6). In general, a finite sequence of these simple transformations cannot completely diagonalize a matrix. There are then two choices: either use the finite sequence of transformations to go most of the way (e.g., to some special form like *tridiagonal* or *Hessenberg*; see §11.3 and §11.6 below) and follow up with the second set of techniques about to be mentioned; or else iterate the finite sequence of simple transformations over and over until the deviation of the matrix from diagonal is negligibly small. This latter approach is conceptually simplest, so we will discuss it in the next section; however, for N greater than ~ 10, it is computationally inefficient by a roughly constant factor ~ 5.

The second set of techniques, called *factorization methods*, is more subtle. Suppose that the matrix $\mathbf{A}$ can be factored into a left factor $\mathbf{F}_L$ and a right factor $\mathbf{F}_R$. Then

$$\mathbf{A} = \mathbf{F}_L \cdot \mathbf{F}_R \qquad \text{or equivalently} \qquad \mathbf{F}_L^{-1} \cdot \mathbf{A} = \mathbf{F}_R \tag{11.0.17}$$

If we now multiply back together the factors in the reverse order and use the second equation in (11.0.17), we get

$$\mathbf{F}_R \cdot \mathbf{F}_L = \mathbf{F}_L^{-1} \cdot \mathbf{A} \cdot \mathbf{F}_L \tag{11.0.18}$$

which we recognize as having effected a similarity transformation on $\mathbf{A}$ with the transformation matrix being $\mathbf{F}_L$! In §11.4 and §11.7 we will discuss the *QR method* that exploits this idea.

Factorization methods also do not converge exactly in a finite number of transformations. But the better ones do converge rapidly and reliably, and, when following an appropriate initial reduction by simple similarity transformations, they are generally the methods of choice.

11.0.4 "Eigenpackages of Canned Eigenroutines"

You have probably gathered by now that the solution of eigensystems is a fairly complicated business. It is. It is one of the few subjects covered in this book for which we do *not* recommend that you avoid canned routines. On the contrary, the purpose of this chapter is precisely to give you some appreciation of what is going on inside such canned routines, so that you can make intelligent choices about using them, and intelligent diagnoses when something goes wrong.

You will find that almost all canned routines in use nowadays trace their ancestry back to routines published in Wilkinson and Reinsch's *Handbook for Automatic Computation, Vol. II, Linear Algebra* [2]. This excellent reference, containing papers by a number of authors, is the Bible of the field. A public-domain implementation of the *Handbook* routines in Fortran is the EISPACK set of programs [3]. The routines in this chapter are translations of either the *Handbook* or EISPACK routines, so understanding these will take you a lot of the way toward understanding those canonical packages.

The successor to EISPACK is LAPACK [4], which also includes the linear algebra routines of LINPACK. This is a Fortran package in which a lot of attention has been paid to efficient execution on modern machines. A C translation is available as CLAPACK.

IMSL [5] and NAG [6] each provide proprietary implementations, in Fortran and C, of what are essentially the *Handbook* routines.

A good "eigenpackage" will provide separate routines, or separate paths through sequences of routines, for the following desired calculations:

- all eigenvalues and no eigenvectors
- all eigenvalues and some corresponding eigenvectors
- all eigenvalues and all corresponding eigenvectors

The purpose of these distinctions is to save compute time and storage; it is wasteful to calculate eigenvectors that you don't need. Often one is interested only in the eigenvectors corresponding to the largest few eigenvalues, or largest few in magnitude, or few that are negative. The method usually used to calculate "some" eigenvectors is typically more efficient than calculating all eigenvectors if you desire fewer than about a quarter of the eigenvectors.

A good eigenpackage also provides separate paths for each of the above calculations for each of the following special forms of the matrix:

- real, symmetric, tridiagonal
- real, symmetric, banded (only a small number of sub- and superdiagonals are nonzero)
- real, symmetric
- real, nonsymmetric
- complex, Hermitian
- complex, non-Hermitian

Again, the purpose of these distinctions is to save time and storage by using the *least* general routine that will serve in any particular application.

In this chapter, as a bare introduction, we give good routines for the following paths:

- all eigenvalues and eigenvectors of a real symmetric, tridiagonal matrix (§11.4)
- all eigenvalues and eigenvectors of a real symmetric, matrix (§11.1 – §11.4)
- all eigenvalues and eigenvectors of a complex Hermitian matrix (§11.5)
- all eigenvalues and eigenvectors of a real nonsymmetric matrix (§11.6 – §11.7)

We also discuss, in §11.8, how to obtain some eigenvectors of general matrices by the method of inverse iteration.

11.0.5 Generalized and Nonlinear Eigenvalue Problems

Many eigenpackages also deal with the so-called *generalized eigenproblem* [7],

$$\mathbf{A} \cdot \mathbf{x} = \lambda \mathbf{B} \cdot \mathbf{x} \tag{11.0.19}$$

where $\mathbf{A}$ and $\mathbf{B}$ are both matrices. Most such problems, where $\mathbf{B}$ is nonsingular, can be handled by the equivalent

$$(\mathbf{B}^{-1} \cdot \mathbf{A}) \cdot \mathbf{x} = \lambda \mathbf{x} \tag{11.0.20}$$

Often $\mathbf{A}$ and $\mathbf{B}$ are symmetric and $\mathbf{B}$ is positive-definite. The matrix $\mathbf{B}^{-1} \cdot \mathbf{A}$ in (11.0.20) is not symmetric, but we can recover a symmetric eigenvalue problem by using the Cholesky decomposition $\mathbf{B} = \mathbf{L} \cdot \mathbf{L}^T$ of §2.9. Multiplying equation (11.0.19) by $\mathbf{L}^{-1}$, we get

$$\mathbf{C} \cdot (\mathbf{L}^T \cdot \mathbf{x}) = \lambda (\mathbf{L}^T \cdot \mathbf{x}) \tag{11.0.21}$$

where

$$\mathbf{C} = \mathbf{L}^{-1} \cdot \mathbf{A} \cdot (\mathbf{L}^{-1})^T \tag{11.0.22}$$

The matrix $\mathbf{C}$ is symmetric and its eigenvalues are the same as those of the original problem (11.0.19); its eigenfunctions are $\mathbf{L}^T \cdot \mathbf{x}$. The efficient way to form $\mathbf{C}$ is first to solve the equation

$$\mathbf{Y} \cdot \mathbf{L}^T = \mathbf{A} \tag{11.0.23}$$

for the lower triangle of the matrix $\mathbf{Y}$. Then solve

$$\mathbf{L} \cdot \mathbf{C} = \mathbf{Y} \tag{11.0.24}$$

for the lower triangle of the symmetric matrix $\mathbf{C}$.

Another generalization of the standard eigenvalue problem is to problems nonlinear in the eigenvalue λ, for example,

$$(\mathbf{A}\lambda^2 + \mathbf{B}\lambda + \mathbf{C}) \cdot \mathbf{x} = 0 \tag{11.0.25}$$

This can be turned into a linear problem by introducing an additional unknown eigenvector $\mathbf{y}$ and solving the $2N \times 2N$ eigensystem

$$\begin{pmatrix} 0 & \mathbf{1} \\ -\mathbf{A}^{-1} \cdot \mathbf{C} & -\mathbf{A}^{-1} \cdot \mathbf{B} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} = \lambda \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} \tag{11.0.26}$$

This technique generalizes to higher-order polynomials in λ. A polynomial of degree M produces a linear $MN \times MN$ eigensystem (see [8]).

11.0.6 Relation to Singular Value Decomposition

The factorization of a matrix $\mathbf{A}$ by the use of its eigenvectors and eigenvalues, equation (11.0.10), seems similar to singular value decomposition (SVD), as was discussed in §2.6. Is it the same thing? In general, no. A first obvious difference is that SVD is not restricted to square matrices, while eigendecomposition is. But what if $\mathbf{A}$ is square? Are the two decompositions then identical?

In general, still no. The difference has to do with what is orthogonal to what. If for a square matrix $\mathbf{A}$ we write the two decompositions (cf. equation 2.6.1 or 2.6.4 and equation 11.0.10),

$$\mathbf{A} = \mathbf{U} \cdot \mathrm{diag}(w_0 \dots w_{N-1}) \cdot \mathbf{V}^T = \mathbf{X}_R \cdot \mathrm{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{X}_L \tag{11.0.27}$$

then for SVD the columns of $\mathbf{U}$ are mutually orthonormal, as are the columns of $\mathbf{V}$. There is no particular orthonormality *between* $\mathbf{U}$ and $\mathbf{V}$. For the eigendecomposition, the situation is the reverse: The rows of $\mathbf{X}_L$ are orthogonal to the columns of $\mathbf{X}_R$ (except for those corresponding to the same eigenvalue), but there is no particular orthogonality among the rows or columns of $\mathbf{X}_L$ or the rows or columns of $\mathbf{X}_R$. The two decompositions in equation (11.0.27) are just, in general, different!

However, the difference disappears when $\mathbf{A}$ is symmetric (or, if complex, Hermitian). In that case, equation (11.0.27) becomes

$$\mathbf{A} = \mathbf{V} \cdot \mathrm{diag}(w_0 \dots w_{N-1}) \cdot \mathbf{V}^T = \mathbf{X}_R \cdot \mathrm{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{X}_R^T \tag{11.0.28}$$

and the fact that each decomposition is unique implies

$$\mathbf{V} = \mathbf{U} = \mathbf{X}_R = \mathbf{X}_L^T \tag{11.0.29}$$

and

$$\lambda_i = w_i, \qquad i = 0, \ldots, N-1 \tag{11.0.30}$$

That is, the (left and right) eigenvectors are the columns of any of the matrices listed in equation (11.0.29), and the corresponding eigenvalues and singular values are identical.

From a general matrix $\mathbf{A}$, not necessarily even square, one can form the two symmetric matrices $\mathbf{A}^T \cdot \mathbf{A}$ and $\mathbf{A} \cdot \mathbf{A}^T$. You can work out from equation (11.0.27) that the eigenvalues of either of these two matrices are squares of singular values of $\mathbf{A}$. However, this doesn't tell you about the eigenvalues of $\mathbf{A}$: The matrix whose eigenvalues are the squares of the eigenvalues of $\mathbf{A}$ is the unrelated matrix $\mathbf{A} \cdot \mathbf{A}$, not $\mathbf{A}^T \cdot \mathbf{A}$ or $\mathbf{A} \cdot \mathbf{A}^T$.

CITED REFERENCES AND FURTHER READING:

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 6.[1]

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[3]

Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, `http://www.netlib.org/lapack`.[4]

IMSL Math/Library Users Manual (Houston: IMSL Inc.), see 2007+, `http://www.vni.com/products/imsl`.[5]

NAG Fortran Library (Oxford, UK: Numerical Algorithms Group), see 2007+, `http://www.nag.co.uk`, Chapter F02.[6]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §7.7.[7]

Wilkinson, J.H. 1965, *The Algebraic Eigenvalue Problem* (New York: Oxford University Press).[8]

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 13.

Horn, R.A., and Johnson, C.R. 1985, *Matrix Analysis* (Cambridge: Cambridge University Press).

11.1 Jacobi Transformations of a Symmetric Matrix

The Jacobi method consists of a sequence of orthogonal similarity transformations of the form of equation (11.0.15). Each transformation (a *Jacobi rotation*) is just a plane rotation designed to annihilate one of the off-diagonal matrix elements. Successive transformations undo previously set zeros, but the off-diagonal elements nevertheless get smaller and smaller, until the matrix is diagonal to machine precision. Accumulating the product of the transformations as you go gives the matrix of

eigenvectors, equation (11.0.16), while the elements of the final diagonal matrix are the eigenvalues.

The Jacobi method is absolutely foolproof for all real symmetric matrices. In particular, it returns the small eigenvalues with better relative accuracy than methods that first reduce the matrix to tridiagonal form. For matrices of order greater than about 10, however, the algorithm is slower, by a significant constant factor, than the QR method we shall give in §11.4. However, the Jacobi algorithm is much simpler than the more efficient methods. We thus recommend it for matrices of moderate order, where expense is not a major consideration.

The basic Jacobi rotation $\mathbf{P}_{pq}$ is a matrix of the form

$$\mathbf{P}_{pq} = \begin{bmatrix} 1 & & & & & & \\ & \cdots & & & & & \\ & & c & \cdots & s & & \\ & & \vdots & 1 & \vdots & & \\ & & -s & \cdots & c & & \\ & & & & & \cdots & \\ & & & & & & 1 \end{bmatrix} \tag{11.1.1}$$

Here all the diagonal elements are unity except for the two elements c in rows (and columns) p and q. All off-diagonal elements are zero except the two elements s and $-s$. The numbers c and s are the cosine and sine of a rotation angle ϕ, so $c^2+s^2 = 1$.

A plane rotation such as (11.1.1) is used to transform the matrix $\mathbf{A}$ according to

$$\mathbf{A}' = \mathbf{P}_{pq}^T \cdot \mathbf{A} \cdot \mathbf{P}_{pq} \tag{11.1.2}$$

Now, $\mathbf{P}_{pq}^T \cdot \mathbf{A}$ changes only rows p and q of $\mathbf{A}$, while $\mathbf{A} \cdot \mathbf{P}_{pq}$ changes only columns p and q. Notice that the subscripts p and q do not denote components of $\mathbf{P}_{pq}$, but rather label which kind of rotation the matrix is, i.e., which rows and columns it affects. Thus the changed elements of $\mathbf{A}$ in (11.1.2) are only in rows p and q, and columns p and q, as indicated below:

$$\mathbf{A}' = \begin{bmatrix} & & a'_{0p} & & a'_{0q} & & \\ & & \vdots & & \vdots & & \\ a'_{p0} & \cdots & a'_{pp} & \cdots & a'_{pq} & \cdots & a'_{p,n-1} \\ & & \vdots & & \vdots & & \\ a'_{q0} & \cdots & a'_{qp} & \cdots & a'_{qq} & \cdots & a'_{q,n-1} \\ & & \vdots & & \vdots & & \\ & & a'_{n-1,p} & & a'_{n-1,q} & & \end{bmatrix} \tag{11.1.3}$$

Multiplying out equation (11.1.2) and using the symmetry of $\mathbf{A}$, we get the explicit formulas

$$\left.\begin{aligned} a'_{rp} &= ca_{rp} - sa_{rq} \\ a'_{rq} &= ca_{rq} + sa_{rp} \end{aligned}\right\} \quad r \neq p,\ r \neq q \tag{11.1.4}$$

$$a'_{pp} = c^2 a_{pp} + s^2 a_{qq} - 2sca_{pq} \tag{11.1.5}$$

$$a'_{qq} = s^2 a_{pp} + c^2 a_{qq} + 2sca_{pq} \tag{11.1.6}$$

$$a'_{pq} = (c^2 - s^2)a_{pq} + sc(a_{pp} - a_{qq}) \tag{11.1.7}$$

The idea of the Jacobi method is to try to zero the off-diagonal elements by a series of plane rotations. Accordingly, to set $a'_{pq} = 0$, equation (11.1.7) gives the following expression for the rotation angle ϕ:

$$\theta \equiv \cot 2\phi \equiv \frac{c^2 - s^2}{2sc} = \frac{a_{qq} - a_{pp}}{2a_{pq}} \tag{11.1.8}$$

If we let $t \equiv s/c$, the definition of θ can be rewritten

$$t^2 + 2t\theta - 1 = 0 \tag{11.1.9}$$

The smaller root of this equation corresponds to a rotation angle less than $\pi/4$ in magnitude; this choice at each stage gives the most stable reduction. Using the form of the quadratic formula with the discriminant in the denominator, we can write this smaller root as

$$t = \frac{\mathrm{sgn}(\theta)}{|\theta| + \sqrt{\theta^2 + 1}} \tag{11.1.10}$$

If θ is so large that θ^2 would overflow on the computer, we set $t = 1/(2\theta)$. It now follows that

$$c = \frac{1}{\sqrt{t^2 + 1}} \tag{11.1.11}$$

$$s = tc \tag{11.1.12}$$

When we actually use equations (11.1.4) – (11.1.7) numerically, we rewrite them to minimize roundoff error. Equation (11.1.7) is replaced by

$$a'_{pq} = 0 \tag{11.1.13}$$

The idea in the remaining equations is to set the new quantity equal to the old quantity plus a small correction. Thus we can use (11.1.7) and (11.1.13) to eliminate a_{qq} from (11.1.5), giving

$$a'_{pp} = a_{pp} - ta_{pq} \tag{11.1.14}$$

Similarly,

$$a'_{qq} = a_{qq} + ta_{pq} \tag{11.1.15}$$

$$a'_{rp} = a_{rp} - s(a_{rq} + \tau a_{rp}) \tag{11.1.16}$$

$$a'_{rq} = a_{rq} + s(a_{rp} - \tau a_{rq}) \tag{11.1.17}$$

where τ $(= \tan\phi/2)$ is defined by

$$\tau \equiv \frac{s}{1 + c} \tag{11.1.18}$$

One can see the convergence of the Jacobi method by considering the sum of the squares of the off-diagonal elements

$$S = \sum_{r \neq s} |a_{rs}|^2 \tag{11.1.19}$$

Equations (11.1.4) – (11.1.7) imply that

$$S' = S - 2|a_{pq}|^2 \tag{11.1.20}$$

(Since the transformation is orthogonal, the sum of the squares of the diagonal elements increases correspondingly by $2|a_{pq}|^2$.) The sequence of S's thus decreases monotonically. Since the sequence is bounded below by zero, and since we can choose a_{pq} to be whatever element we want, the sequence can be made to converge to zero.

Eventually one obtains a matrix $\mathbf{D}$ that is diagonal to machine precision. The diagonal elements give the eigenvalues of the original matrix $\mathbf{A}$, since

$$\mathbf{D} = \mathbf{V}^T \cdot \mathbf{A} \cdot \mathbf{V} \tag{11.1.21}$$

where

$$\mathbf{V} = \mathbf{P}_1 \cdot \mathbf{P}_2 \cdot \mathbf{P}_3 \cdots \tag{11.1.22}$$

the $\mathbf{P}_i$'s being the successive Jacobi rotation matrices. The columns of $\mathbf{V}$ are the eigenvectors (since $\mathbf{A} \cdot \mathbf{V} = \mathbf{V} \cdot \mathbf{D}$). They can be computed by applying

$$\mathbf{V}' = \mathbf{V} \cdot \mathbf{P}_i \tag{11.1.23}$$

at each stage of calculation, where initially $\mathbf{V}$ is the identity matrix. In detail, equation (11.1.23) is

$$\begin{aligned} v'_{rs} &= v_{rs} \qquad (s \neq p,\ s \neq q) \\ v'_{rp} &= c v_{rp} - s v_{rq} \\ v'_{rq} &= s v_{rp} + c v_{rq} \end{aligned} \tag{11.1.24}$$

We rewrite these equations in terms of τ as in equations (11.1.16) and (11.1.17) to minimize roundoff.

The only remaining question is the strategy one should adopt for the order in which the elements are to be annihilated. Jacobi's original algorithm of 1846 searched the whole upper triangle at each stage and set the largest off-diagonal element to zero. This is a reasonable strategy for hand calculation, but it is prohibitive on a computer since the search alone makes each Jacobi rotation a process of order N^2 instead of N.

A better strategy for our purposes is the *cyclic Jacobi method*, where one annihilates elements in strict order. For example, one can simply proceed down the rows: $\mathbf{P}_{01}, \mathbf{P}_{02}, ..., \mathbf{P}_{0,n-1}$; then $\mathbf{P}_{12}, \mathbf{P}_{13}$, etc. One can show that convergence is generally quadratic for either the original or the cyclic Jacobi method, for nondegenerate eigenvalues. One such set of $n(n-1)/2$ Jacobi rotations is called a *sweep*.

The program below, based on the implementations in [1,2], uses two further refinements:

- In the first three sweeps, we carry out the pq rotation only if $|a_{pq}| > \epsilon$ for some threshold value
$$\epsilon = \frac{1}{5}\frac{S_0}{n^2} \tag{11.1.25}$$
where S_0 is the sum of the off-diagonal moduli,
$$S_0 = \sum_{r<s} |a_{rs}| \tag{11.1.26}$$

- After four sweeps, if $|a_{pq}| \ll |a_{pp}|$ and $|a_{pq}| \ll |a_{qq}|$, we set $|a_{pq}| = 0$ and skip the rotation. The criterion used in the comparison is $|a_{pq}| < 10^{-(D+2)}|a_{pp}|$, where D is the number of significant decimal digits on the machine, and similarly for $|a_{qq}|$.

Typical matrices require six to ten sweeps to achieve convergence, or $3n^2$ to $5n^2$ Jacobi rotations. Each rotation requires of order $8n$ floating-point operations, so the total labor is of order $24n^3$ to $40n^3$ operations. Calculation of the eigenvectors as well as the eigenvalues changes the operation count from $8n$ to $12n$ per rotation, which is only a 50% overhead.

The following routine implements the Jacobi method. Simply create a `Jacobi` object using your symmetric matrix `a[0..n-1] [0..n-1]`:

```
Jacobi jac(a);
```

The vector `d[0..n-1]` then contains the eigenvalues of `a`. During the computation, it contains the current diagonal of `a`. The matrix `v[0..n-1][0..n-1]` outputs the normalized eigenvector belonging to `d[k]` in column `k`. The parameter `nrot` is the number of Jacobi rotations that were needed to achieve convergence.

eigen_sym.h

```
struct Jacobi {
```

Computes all eigenvalues and eigenvectors of a real symmetric matrix by Jacobi's method.

```
    const Int n;
    MatDoub a,v;
    VecDoub d;
    Int nrot;
    const Doub EPS;

    Jacobi(MatDoub_I &aa) : n(aa.nrows()), a(aa), v(n,n), d(n), nrot(0),
        EPS(numeric_limits<Doub>::epsilon())
```

Computes all eigenvalues and eigenvectors of a real symmetric matrix `a[0..n-1][0..n-1]`. On output, `d[0..n-1]` contains the eigenvalues of `a` sorted into descending order, while `v[0..n-1][0..n-1]` is a matrix whose columns contain the corresponding normalized eigenvectors. `nrot` contains the number of Jacobi rotations that were required. Only the upper triangle of `a` is accessed.

```
    {
        Int i,j,ip,iq;
        Doub tresh,theta,tau,t,sm,s,h,g,c;
        VecDoub b(n),z(n);
        for (ip=0;ip<n;ip++) {
            for (iq=0;iq<n;iq++) v[ip][iq]=0.0;
            v[ip][ip]=1.0;
        }
        for (ip=0;ip<n;ip++) {
            b[ip]=d[ip]=a[ip][ip];
            z[ip]=0.0;
        }
        for (i=1;i<=50;i++) {
            sm=0.0;
            for (ip=0;ip<n-1;ip++) {
                for (iq=ip+1;iq<n;iq++)
                    sm += abs(a[ip][iq]);
            }
            if (sm == 0.0) {
                eigsrt(d,&v);
                return;
            }
            if (i < 4)
                tresh=0.2*sm/(n*n);
```

Initialize to the identity matrix.

Initialize `b` and `d` to the diagonal of `a`.

This vector will accumulate terms of the form ta_{pq} as in equation (11.1.14).

Sum magnitude of off-diagonal elements.

The normal return, which relies on quadratic convergence to machine underflow.

On the first three sweeps...

```
            else
                tresh=0.0;                                  ...thereafter.
            for (ip=0;ip<n-1;ip++) {
                for (iq=ip+1;iq<n;iq++) {
                    g=100.0*abs(a[ip][iq]);
                    After four sweeps, skip the rotation if the off-diagonal element is small.
                    if (i > 4 && g <= EPS*abs(d[ip]) && g <= EPS*abs(d[iq]))
                            a[ip][iq]=0.0;
                    else if (abs(a[ip][iq]) > tresh) {
                        h=d[iq]-d[ip];
                        if (g <= EPS*abs(h))
                            t=(a[ip][iq])/h;                t = 1/(2θ)
                        else {
                            theta=0.5*h/(a[ip][iq]);        Equation (11.1.10).
                            t=1.0/(abs(theta)+sqrt(1.0+theta*theta));
                            if (theta < 0.0) t = -t;
                        }
                        c=1.0/sqrt(1+t*t);
                        s=t*c;
                        tau=s/(1.0+c);
                        h=t*a[ip][iq];
                        z[ip] -= h;
                        z[iq] += h;
                        d[ip] -= h;
                        d[iq] += h;
                        a[ip][iq]=0.0;
                        for (j=0;j<ip;j++)                  Case of rotations 0 ≤ j < p.
                            rot(a,s,tau,j,ip,j,iq);
                        for (j=ip+1;j<iq;j++)               Case of rotations p < j < q.
                            rot(a,s,tau,ip,j,j,iq);
                        for (j=iq+1;j<n;j++)                Case of rotations q < j < n.
                            rot(a,s,tau,ip,j,iq,j);
                        for (j=0;j<n;j++)
                            rot(v,s,tau,j,ip,j,iq);
                        ++nrot;
                    }
                }
            }
            for (ip=0;ip<n;ip++) {
                b[ip] += z[ip];
                d[ip]=b[ip];                                Update d with the sum of ta_pq,
                z[ip]=0.0;                                  and reinitialize z.
            }
        }
        throw("Too many iterations in routine jacobi");
    }
    inline void rot(MatDoub_IO &a, const Doub s, const Doub tau, const Int i,
        const Int j, const Int k, const Int l)
    {
        Doub g=a[i][j];
        Doub h=a[k][l];
        a[i][j]=g-s*(h+g*tau);
        a[k][l]=h+s*(g-h*tau);
    }
};
```

Note that the above routine assumes that underflows are set to zero. On machines where this is not true, the program must be modified. See §1.5.4 and/or find out about the `fesetenv` (Linux) or `__controlfp` (Microsoft) functions.

The Jacobi method does not order the eigenvalues itself. We incorporate the following routine to sort the eigenvalues into descending order. The same routine is used in `Symmeig` in the next section. (The method, straight insertion, is N^2 rather

than $N \log N$; but since you have just done an N^3 procedure to get the eigenvalues, you can afford yourself this little indulgence.)

eigen_sym.h

```
void eigsrt(VecDoub_IO &d, MatDoub_IO *v=NULL)
```

Given the eigenvalues `d[0..n-1]` and (optionally) the eigenvectors `v[0..n-1][0..n-1]` as determined by `Jacobi` (§11.1) or `tqli` (§11.4), this routine sorts the eigenvalues into descending order and rearranges the columns of `v` correspondingly. The method is straight insertion.

```
{
    Int k;
    Int n=d.size();
    for (Int i=0;i<n-1;i++) {
        Doub p=d[k=i];
        for (Int j=i;j<n;j++)
            if (d[j] >= p) p=d[k=j];
        if (k != i) {
            d[k]=d[i];
            d[i]=p;
            if (v != NULL)
                for (Int j=0;j<n;j++) {
                    p=(*v)[j][i];
                    (*v)[j][i]=(*v)[j][k];
                    (*v)[j][k]=p;
                }
        }
    }
}
```

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §8.4.

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[1]

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

11.2 Real Symmetric Matrices

As already mentioned, the optimum strategy in most cases for finding eigenvalues and eigenvectors is, first, to reduce the matrix to a simple form, only then beginning an iterative procedure. For symmetric matrices, the preferred simple form is tridiagonal.

Here is a routine based on this strategy that finds all eigenvalues and eigenvectors of a real symmetric matrix. It is typically a factor of about five faster than the Jacobi routine of the previous section. The implementations of the functions `tred2` and `tqli` that reduce the matrix to tridiagonal form and then find the eigensystem are discussed in the next two sections.

There are two user interfaces, implemented as two constructors. The first constructor is the usual one:

```
Symmeig s(a);
```

It returns the eigenvalues of a in descending order in s.d[0..n-1]. The normalized eigenvector corresponding to d[k] is in the matrix column s.z[0..n-1][k]. Setting the default argument to false suppresses the computation of the eigenvectors:

```
Symmeig s(a,false);
```

If you already have a matrix in tridiagonal form, you use the other constructor, which accepts the diagonal and subdiagonal of the matrix as vectors:

```
Symmeig s(d,e);
```

Again, you can suppress the computation of eigenvectors by setting the default argument to false.

Here is the routine:

eigen_sym.h

```
struct Symmeig {
```
Computes all eigenvalues and eigenvectors of a real symmetric matrix by reduction to tridiagonal form followed by QL iteration.
```
    Int n;
    MatDoub z;
    VecDoub d,e;
    Bool yesvecs;

    Symmeig(MatDoub_I &a, Bool yesvec=true) : n(a.nrows()), z(a), d(n),
        e(n), yesvecs(yesvec)
```
    Computes all eigenvalues and eigenvectors of a real symmetric matrix a[0..n-1][0..n-1] by reduction to tridiagonal form followed by QL iteration. On output, d[0..n-1] contains the eigenvalues of a sorted into descending order, while z[0..n-1][0..n-1] is a matrix whose columns contain the corresponding normalized eigenvectors. If yesvecs is input as true (the default), then the eigenvectors are computed. If yesvecs is input as false, only the eigenvalues are computed.
```
    {
        tred2();                    Reduction to tridiagonal form; see §11.3.
        tqli();                     Eigensystem of tridiagonal matrix; see §11.4.
        sort();
    }
    Symmeig(VecDoub_I &dd, VecDoub_I &ee, Bool yesvec=true) :
        n(dd.size()), d(dd), e(ee), z(n,n,0.0), yesvecs(yesvec)
```
    Computes all eigenvalues and (optionally) eigenvectors of a real, symmetric, tridiagonal matrix by QL iteration. On input, dd[0..n-1] contains the diagonal elements of the tridiagonal matrix. The vector ee[0..n-1] inputs the subdiagonal elements of the tridiagonal matrix, with ee[0] arbitrary. Output is the same as the constructor above.
```
    {
        for (Int i=0;i<n;i++) z[i][i]=1.0;
        tqli();
        sort();
    }
    void sort() {
        if (yesvecs)
            eigsrt(d,&z);
        else
            eigsrt(d);
    }
    void tred2();
    void tqli();
    Doub pythag(const Doub a, const Doub b);
};
```

11.3 Reduction of a Symmetric Matrix to Tridiagonal Form: Givens and Householder Reductions

The previous section outlined the grand strategy of (i) reduction to tridiagonal form, followed by (ii) finding the eigenvalues and eigenvectors of a tridiagonal matrix. In this section we implement the first of these steps.

11.3.1 Givens Method

The *Givens reduction* is a modification of the Jacobi method. Instead of trying to reduce the matrix all the way to diagonal form, we are content to stop when the matrix is tridiagonal. This allows the procedure to be carried out *in a finite number of steps*, unlike the Jacobi method, which requires iteration to convergence.

For the Givens method, we choose the rotation angle in equation (11.1.1) so as to zero an element that is *not* at one of the four "corners," i.e., not a_{pp}, a_{pq}, or a_{qq} in equation (11.1.3). Specifically, we first choose $\mathbf{P}_{12}$ to annihilate a_{20} (and, by symmetry, a_{02}). Then we choose $\mathbf{P}_{13}$ to annihilate a_{30}. In general, we choose the sequence

$$\mathbf{P}_{12}, \mathbf{P}_{13}, \ldots, \mathbf{P}_{1,n-1}; \mathbf{P}_{23}, \ldots, \mathbf{P}_{2,n-1}; \ldots; \mathbf{P}_{n-2,n-1}$$

where $\mathbf{P}_{jk}$ annihilates $a_{k,j-1}$. The method works because elements such as a'_{rp} and a'_{rq}, with $r \neq p$ $r \neq q$, are linear combinations of the old quantities a_{rp} and a_{rq}, by equation (11.1.4). Thus, if a_{rp} and a_{rq} have already been set to zero, they remain zero as the reduction proceeds. Evidently, of order $n^2/2$ rotations are required, and the number of multiplications in a straightforward implementation is of order $4n^3/3$, not counting those for keeping track of the product of the transformation matrices, required for the eigenvectors.

The Householder method, to be discussed next, is just as stable as the Givens reduction and it is a factor of two more efficient, so the Givens method is not generally used. However, the Givens reduction can be reformulated to reduce the number of operations by a factor of two, and also avoid the necessity of taking square roots [1]. This appears to make the algorithm competitive with the Householder reduction. Unfortunately, this "fast Givens" reduction has to be monitored to avoid overflows, and the variables have to be periodically rescaled. There does not seem to be any compelling reason to prefer the Givens reduction over the Householder method.

11.3.2 Householder Method

The Householder algorithm reduces an $n \times n$ symmetric matrix $\mathbf{A}$ to tridiagonal form by $n - 2$ orthogonal transformations. Each transformation annihilates the required part of a whole column and whole corresponding row. The basic ingredient is a Householder matrix $\mathbf{P}$, which has the form

$$\mathbf{P} = \mathbf{1} - 2\mathbf{w} \cdot \mathbf{w}^T \tag{11.3.1}$$

where $\mathbf{w}$ is a real vector with $|\mathbf{w}|^2 = 1$. (In the present notation, the *outer* or matrix product of two vectors, $\mathbf{a}$ and $\mathbf{b}$, is written $\mathbf{a} \cdot \mathbf{b}^T$, while the *inner* or scalar product

of the vectors is written as $\mathbf{a}^T \cdot \mathbf{b}$.) The matrix $\mathbf{P}$ is orthogonal, because

$$\begin{aligned} \mathbf{P}^2 &= (\mathbf{1} - 2\mathbf{w} \cdot \mathbf{w}^T) \cdot (\mathbf{1} - 2\mathbf{w} \cdot \mathbf{w}^T) \\ &= \mathbf{1} - 4\mathbf{w} \cdot \mathbf{w}^T + 4\mathbf{w} \cdot (\mathbf{w}^T \cdot \mathbf{w}) \cdot \mathbf{w}^T \\ &= \mathbf{1} \end{aligned} \tag{11.3.2}$$

Therefore $\mathbf{P} = \mathbf{P}^{-1}$. But $\mathbf{P}^T = \mathbf{P}$, and so $\mathbf{P}^T = \mathbf{P}^{-1}$, proving orthogonality.

Rewrite $\mathbf{P}$ as

$$\mathbf{P} = \mathbf{1} - \frac{\mathbf{u} \cdot \mathbf{u}^T}{H} \tag{11.3.3}$$

where the scalar H is

$$H \equiv \tfrac{1}{2} |\mathbf{u}|^2 \tag{11.3.4}$$

and $\mathbf{u}$ can now be any vector. Suppose $\mathbf{x}$ is the vector composed of the first column of $\mathbf{A}$. Choose

$$\mathbf{u} = \mathbf{x} \mp |\mathbf{x}| \mathbf{e}_0 \tag{11.3.5}$$

where $\mathbf{e}_0$ is the unit vector $[1, 0, \ldots, 0]^T$ and the choice of signs will be made later. Then

$$\begin{aligned} \mathbf{P} \cdot \mathbf{x} &= \mathbf{x} - \frac{\mathbf{u}}{H} \cdot (\mathbf{x} \mp |\mathbf{x}| \mathbf{e}_0)^T \cdot \mathbf{x} \\ &= \mathbf{x} - \frac{2\mathbf{u} \cdot (|\mathbf{x}|^2 \mp |\mathbf{x}| x_0)}{2|\mathbf{x}|^2 \mp 2|\mathbf{x}| x_0} \\ &= \mathbf{x} - \mathbf{u} \\ &= \pm |\mathbf{x}| \mathbf{e}_0 \end{aligned} \tag{11.3.6}$$

This shows that the Householder matrix $\mathbf{P}$ acts on a given vector $\mathbf{x}$ to zero all its elements except the first one.

To reduce a symmetric matrix $\mathbf{A}$ to tridiagonal form, we choose the vector $\mathbf{x}$ for the first Householder matrix to be the lower $n-1$ elements of column 0. Then the lower $n-2$ elements will be zeroed:

$$\mathbf{P}_1 \cdot \mathbf{A} = \left[\begin{array}{c|ccc} 1 & 0 & \cdots & 0 \\ \hline 0 & & & \\ \vdots & & {}^{(n-1)}\mathbf{P}_1 & \\ 0 & & & \end{array}\right] \cdot \left[\begin{array}{c|ccc} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ \hline a_{10} & & & \\ a_{20} & & & \\ \vdots & & \text{irrelevant} & \\ a_{n-1,0} & & & \end{array}\right]$$

$$= \left[\begin{array}{c|ccc} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ \hline k & & & \\ 0 & & & \\ \vdots & & \text{irrelevant} & \\ 0 & & & \end{array}\right] \tag{11.3.7}$$

Here we have written the matrices in partitioned form, with ${}^{(n-1)}\mathbf{P}$ denoting a Householder matrix with dimensions $(n-1)\times(n-1)$. The quantity k is simply plus or minus the magnitude of the vector $[a_{10},\ldots,a_{n-1,0}]^T$.

The complete orthogonal transformation is now

$$\mathbf{A}' = \mathbf{P}\cdot\mathbf{A}\cdot\mathbf{P} = \left[\begin{array}{c|ccc} a_{00} & k & 0\ \cdots & 0 \\ \hline k & & & \\ 0 & & & \\ \vdots & & \text{irrelevant} & \\ 0 & & & \end{array}\right] \tag{11.3.8}$$

We have used the fact that $\mathbf{P}^T = \mathbf{P}$.

Now choose the vector $\mathbf{x}$ for the second Householder matrix to be the bottom $n-2$ elements of column 1, and from it construct

$$\mathbf{P}_2 \equiv \left[\begin{array}{cc|ccc} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \hline 0 & 0 & & & \\ \vdots & \vdots & & {}^{(n-2)}\mathbf{P}_2 & \\ 0 & 0 & & & \end{array}\right] \tag{11.3.9}$$

The identity block in the upper-left corner ensures that the tridiagonalization achieved in the first step will not be spoiled by this one, while the $(n-2)$-dimensional Householder matrix ${}^{(n-2)}\mathbf{P}_2$ creates one additional row and column of the tridiagonal output. Clearly, a sequence of $n-2$ such transformations will reduce the matrix $\mathbf{A}$ to tridiagonal form.

Instead of actually carrying out the matrix multiplications in $\mathbf{P}\cdot\mathbf{A}\cdot\mathbf{P}$, we compute a vector

$$\mathbf{p} \equiv \frac{\mathbf{A}\cdot\mathbf{u}}{H} \tag{11.3.10}$$

Then

$$\mathbf{A}\cdot\mathbf{P} = \mathbf{A}\cdot\left(1 - \frac{\mathbf{u}\cdot\mathbf{u}^T}{H}\right) = \mathbf{A} - \mathbf{p}\cdot\mathbf{u}^T$$

$$\mathbf{A}' = \mathbf{P}\cdot\mathbf{A}\cdot\mathbf{P} = \mathbf{A} - \mathbf{p}\cdot\mathbf{u}^T - \mathbf{u}\cdot\mathbf{p}^T + 2K\mathbf{u}\cdot\mathbf{u}^T$$

where the scalar K is defined by

$$K = \frac{\mathbf{u}^T\cdot\mathbf{p}}{2H} \tag{11.3.11}$$

If we write

$$\mathbf{q} \equiv \mathbf{p} - K\mathbf{u} \tag{11.3.12}$$

then we have

$$\mathbf{A}' = \mathbf{A} - \mathbf{q}\cdot\mathbf{u}^T - \mathbf{u}\cdot\mathbf{q}^T \tag{11.3.13}$$

This is the computationally useful formula.

Following [2], the routine for Householder reduction given below actually starts in the column $n-1$ of $\mathbf{A}$, not column 0 as in the explanation above. In detail, the equations are as follows: At stage m ($m = 1, 2, \ldots, n-2$), the vector $\mathbf{u}$ has the form

$$\mathbf{u}^T = [a_{i0}, a_{i1}, \ldots, a_{i,i-2}, a_{i,i-1} \pm \sqrt{\sigma}, 0, \ldots, 0] \tag{11.3.14}$$

Here

$$i \equiv n - m = n - 1, n - 2, \ldots, 2 \tag{11.3.15}$$

and the quantity σ ($|x|^2$ in our earlier notation) is

$$\sigma = (a_{i0})^2 + \cdots + (a_{i,i-1})^2 \tag{11.3.16}$$

Choose the sign of $\sqrt{\sigma}$ in (11.3.14) to be the same as the sign of $a_{i,i-1}$ to lessen roundoff error.

Variables are thus computed in the following order: $\sigma, \mathbf{u}, H, \mathbf{p}, K, \mathbf{q}, \mathbf{A}'$. At any stage m, $\mathbf{A}$ is tridiagonal in its last $m - 1$ rows and columns.

No extra storage arrays are needed for the intermediate results. At stage m, the vectors $\mathbf{p}$ and $\mathbf{q}$ are nonzero only in elements $0, \ldots, i$ (recall that $i = n - m$), while $\mathbf{u}$ is nonzero only in elements $0, \ldots, i - 1$. The elements of the vector `e` are being determined in the order $n - 1, n - 2, \ldots$, so we can store $\mathbf{p}$ in the elements of `e` not already determined. The vector $\mathbf{q}$ can overwrite $\mathbf{p}$ once $\mathbf{p}$ is no longer needed. We store $\mathbf{u}$ in row i of `a` and $\mathbf{u}/H$ in column i of `a`. Once the reduction is complete, we compute the matrices $\mathbf{Q}_j$ using the quantities $\mathbf{u}$ and $\mathbf{u}/H$ that have been stored in `a`. Since $\mathbf{Q}_j$ is an identity matrix from row and column $n - j$ on, we only need compute its elements up to row and column $n - j - 1$. These can overwrite the $\mathbf{u}$'s and $\mathbf{u}/H$'s in the corresponding rows and columns of `a`, which are no longer required for subsequent $\mathbf{Q}$'s.

The routine `tred2`, given below, includes one further refinement. If the quantity σ is zero or "small" at any stage, one can skip the corresponding transformation. A simple criterion, such as

$$\sigma < \frac{\text{smallest positive number representable on machine}}{\text{machine precision}}$$

would be fine most of the time. A more careful criterion is actually used. At stage i, define the quantity

$$\epsilon = \sum_{k=0}^{i-1} |a_{ik}| \tag{11.3.17}$$

If $\epsilon = 0$ to machine precision, we skip the transformation. Otherwise we redefine

$$a_{ik} \quad \text{becomes} \quad a_{ik}/\epsilon \tag{11.3.18}$$

and use the scaled variables for the transformation. (A Householder transformation depends only on the ratios of the elements.)

If the eigenvectors of the final tridiagonal matrix are found (for example, by the routine in the next section), then the eigenvectors of $\mathbf{A}$ can be obtained by applying the accumulated transformation

$$\mathbf{Q} = \mathbf{P}_1 \cdot \mathbf{P}_2 \cdots \mathbf{P}_{n-2} \tag{11.3.19}$$

to those eigenvectors. We therefore form $\mathbf{Q}$ by recursion after all the $\mathbf{P}$'s have been determined:

$$\begin{aligned} \mathbf{Q}_{n-2} &= \mathbf{P}_{n-2} \\ \mathbf{Q}_j &= \mathbf{P}_j \cdot \mathbf{Q}_{j+1}, \qquad j = n-3, \ldots, 1 \\ \mathbf{Q} &= \mathbf{Q}_1 \end{aligned} \tag{11.3.20}$$

Input for the routine below is the real symmetric matrix $\mathbf{A}$ stored in the matrix `z[0..n-1][0..n-1]`. On output, `z` contains the elements of the orthogonal matrix $\mathbf{Q}$. The vector `d[0..n-1]` is set to the diagonal elements of the tridiagonal matrix $\mathbf{A}'$, while the vector `e[0..n-1]` is set to the off-diagonal elements in its components `1` through `n-1`, with `e[0]=0`.

Note that when dealing with a matrix whose elements vary over many orders of magnitude, it is desirable that the matrix be permuted, insofar as possible, so that the smaller elements are in the top left-hand corner. This is because the reduction is performed starting from the bottom right-hand corner, and a mixture of small and large elements there can lead to considerable rounding errors.

In the limit of large n, the operation count of the Householder reduction is $4n^3/3$ for eigenvalues only, and $8n^3/3$ for both eigenvalues and eigenvectors. The routine `tred2` is designed for use with the routine `tqli` of the next section. `tqli` finds the eigenvalues and eigenvectors of a symmetric tridiagonal matrix. For many years, the combination of `tred2` and `tqli` was the most efficient known technique for finding all the eigenvalues and eigenvectors (or just all the eigenvalues) of a real symmetric matrix. For moderate-sized matrices, it is still competitive with newer, more complicated methods.

eigen_sym.h

`void Symmeig::tred2()`

Householder reduction of a real symmetric matrix `z[0..n-1][0..n-1]`. (The input matrix $\mathbf{A}$ to `Symmeig` is stored in `z`.) On output, `z` is replaced by the orthogonal matrix $\mathbf{Q}$ effecting the transformation. `d[0..n-1]` contains the diagonal elements of the tridiagonal matrix and `e[0..n-1]` the off-diagonal elements, with `e[0]=0`. If `yesvecs` is `false`, so that only eigenvalues will subsequently be determined, several statements are omitted, in which case `z` contains no useful information on output.

```
{
    Int l,k,j,i;
    Doub scale,hh,h,g,f;
    for (i=n-1;i>0;i--) {
        l=i-1;
        h=scale=0.0;
        if (l > 0) {
            for (k=0;k<i;k++)
                scale += abs(z[i][k]);
            if (scale == 0.0)                   Skip transformation.
                e[i]=z[i][l];
            else {
                for (k=0;k<i;k++) {
                    z[i][k] /= scale;           Use scaled a's for transformation.
                    h += z[i][k]*z[i][k];       Form σ in h.
                }
                f=z[i][l];
                g=(f >= 0.0 ? -sqrt(h) : sqrt(h));
                e[i]=scale*g;
                h -= f*g;                       Now h is equation (11.3.4).
                z[i][l]=f-g;                    Store u in row i of z.
                f=0.0;
                for (j=0;j<i;j++) {
                    if (yesvecs)                Store u/H in column i of z.
                        z[j][i]=z[i][j]/h;
                    g=0.0;                      Form an element of A · u in g.
                    for (k=0;k<j+1;k++)
                        g += z[j][k]*z[i][k];
                    for (k=j+1;k<i;k++)
                        g += z[k][j]*z[i][k];
                    e[j]=g/h;                   Form element of p in temporarily unused
                    f += e[j]*z[i][j];              element of e.
                }
                hh=f/(h+h);                     Form K, equation (11.3.11).
                for (j=0;j<i;j++) {             Form q and store in e overwriting p.
                    f=z[i][j];
                    e[j]=g=e[j]-hh*f;
                    for (k=0;k<j+1;k++)         Reduce z, equation (11.3.13).
```

```
                    z[j][k] -= (f*e[k]+g*z[i][k]);
                }
            }
        } else
            e[i]=z[i][l];
        d[i]=h;
    }
    if (yesvecs) d[0]=0.0;
    e[0]=0.0;
    for (i=0;i<n;i++) {                          Begin accumulation of transformation ma-
        if (yesvecs) {                               trices.
            if (d[i] != 0.0) {                   This block skipped when i=0.
                for (j=0;j<i;j++) {
                    g=0.0;
                    for (k=0;k<i;k++)            Use u and u/H stored in z to form P·Q.
                        g += z[i][k]*z[k][j];
                    for (k=0;k<i;k++)
                        z[k][j] -= g*z[k][i];
                }
            }
            d[i]=z[i][i];
            z[i][i]=1.0;                         Reset row and column of z to identity
            for (j=0;j<i;j++) z[j][i]=z[i][j]=0.0;    matrix for next iteration.
        } else {
            d[i]=z[i][i];                        Only this statement remains.
        }
    }
}
```

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §5.1.[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

11.4 Eigenvalues and Eigenvectors of a Tridiagonal Matrix

We now turn to the second step in the grand strategy outlined in §11.2, namely computing the eigenvectors and eigenvalues of a tridiagonal matrix.

11.4.1 Evaluation of the Characteristic Polynomial

Once our original real symmetric matrix has been reduced to tridiagonal form, one possible way to determine its eigenvalues is to find the roots of the characteristic polynomial $p_n(\lambda)$ directly. The characteristic polynomial of a tridiagonal matrix can be evaluated for any trial value of λ by an efficient recursion relation (see [1], for example). The polynomials of lower degree produced during the recurrence form a Sturmian sequence that can be used to localize the eigenvalues to intervals on the

real axis. A root-finding method such as bisection or Newton's method can then be employed to refine the intervals. The corresponding eigenvectors can then be found by inverse iteration (see §11.8).

Procedures based on these ideas can be found in [2,3]. If, however, more than a small fraction of all the eigenvalues and eigenvectors is required, then the factorization method next considered is much more efficient.

11.4.2 The QR and QL Algorithms

The basic idea behind the QR algorithm is that any real matrix can be decomposed in the form

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \tag{11.4.1}$$

where $\mathbf{Q}$ is orthogonal and $\mathbf{R}$ is upper triangular. For a general matrix, the decomposition is constructed by applying Householder transformations to annihilate successive columns of $\mathbf{A}$ below the diagonal (see §2.10).

Now consider the matrix formed by writing the factors in (11.4.1) in the opposite order:

$$\mathbf{A}' = \mathbf{R} \cdot \mathbf{Q} \tag{11.4.2}$$

Since $\mathbf{Q}$ is orthogonal, equation (11.4.1) gives $\mathbf{R} = \mathbf{Q}^T \cdot \mathbf{A}$. Thus equation (11.4.2) becomes

$$\mathbf{A}' = \mathbf{Q}^T \cdot \mathbf{A} \cdot \mathbf{Q} \tag{11.4.3}$$

We see that $\mathbf{A}'$ is an orthogonal transformation of $\mathbf{A}$.

You can verify that a QR transformation preserves the following properties of a matrix: symmetry, tridiagonal form, and Hessenberg form (to be defined in §11.6).

There is nothing special about choosing one of the factors of $\mathbf{A}$ to be upper triangular; one could equally well make it lower triangular. This is called the QL algorithm, since

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{L} \tag{11.4.4}$$

where $\mathbf{L}$ is lower triangular. (The standard, but confusing, nomenclature R and L stands for whether the *right* or *left* of the matrix is nonzero.)

Recall that in the Householder reduction to tridiagonal form in §11.3, we started in column $n-1$ of the original matrix. To minimize roundoff, we then exhorted you to put the biggest elements of the matrix in the lower right-hand corner, if you can. If we now wish to diagonalize the resulting tridiagonal matrix, the QL algorithm will have smaller roundoff than the QR algorithm, so we shall use QL henceforth.

The QL algorithm consists of a *sequence* of orthogonal transformations:

$$\begin{aligned} \mathbf{A}_s &= \mathbf{Q}_s \cdot \mathbf{L}_s \\ \mathbf{A}_{s+1} &= \mathbf{L}_s \cdot \mathbf{Q}_s \qquad (= \mathbf{Q}_s^T \cdot \mathbf{A}_s \cdot \mathbf{Q}_s) \end{aligned} \tag{11.4.5}$$

The following (nonobvious!) theorem is the basis of the algorithm for a general matrix $\mathbf{A}$: (i) If $\mathbf{A}$ has eigenvalues of different absolute value $|\lambda_i|$, then $\mathbf{A}_s \to$ [lower triangular form] as $s \to \infty$. The eigenvalues appear on the diagonal in increasing order of absolute magnitude. (ii) If $\mathbf{A}$ has an eigenvalue $|\lambda_i|$ of multiplicity p, $\mathbf{A}_s \to$ [lower triangular form] as $s \to \infty$, except for a diagonal block matrix of

order p, whose eigenvalues $\to \lambda_i$. The proof of this theorem is fairly lengthy; see, for example, [4].

The workload in the QL algorithm is $O(n^3)$ per iteration for a general matrix, which is prohibitive. However, the workload is only $O(n)$ per iteration for a tridiagonal matrix and $O(n^2)$ for a Hessenberg matrix, which makes it highly efficient on these forms.

In this section we are concerned only with the case where $\mathbf{A}$ is a real, symmetric, tridiagonal matrix. All the eigenvalues λ_i are thus real. According to the theorem, if any λ_i has a multiplicity p, then there must be at least $p-1$ zeros on the sub- and superdiagonals. Thus the matrix can be split into submatrices that can be diagonalized separately, and the complication of diagonal blocks that can arise in the general case is irrelevant.

In the proof of the theorem quoted above, one finds that in general a superdiagonal element converges to zero like

$$a_{ij}^{(s)} \sim \left(\frac{\lambda_i}{\lambda_j}\right)^s \tag{11.4.6}$$

Although $\lambda_i < \lambda_j$, convergence can be slow if λ_i is close to λ_j. Convergence can be accelerated by the technique of *shifting*: If k is any constant, then $\mathbf{A} - k\mathbf{1}$ has eigenvalues $\lambda_i - k$. If we decompose

$$\mathbf{A}_s - k_s\mathbf{1} = \mathbf{Q}_s \cdot \mathbf{L}_s \tag{11.4.7}$$

so that

$$\begin{aligned} \mathbf{A}_{s+1} &= \mathbf{L}_s \cdot \mathbf{Q}_s + k_s\mathbf{1} \\ &= \mathbf{Q}_s^T \cdot \mathbf{A}_s \cdot \mathbf{Q}_s \end{aligned} \tag{11.4.8}$$

then the convergence is determined by the ratio

$$\frac{\lambda_i - k_s}{\lambda_j - k_s} \tag{11.4.9}$$

The idea is to choose the shift k_s at each stage to maximize the rate of convergence. A good choice for the shift initially would be k_s close to λ_0, the smallest eigenvalue. Then the first row of off-diagonal elements would tend rapidly to zero. However, λ_0 is not usually known a priori. A very effective strategy in practice (although there is no proof that it is optimal) is to compute the eigenvalues of the leading 2×2 diagonal submatrix of $\mathbf{A}$. Then set k_s equal to the eigenvalue closer to a_{00}.

More generally, suppose you have already found r eigenvalues of $\mathbf{A}$. Then you can *deflate* the matrix by crossing out the first r rows and columns, leaving

$$\mathbf{A} = \begin{bmatrix} 0 & & & \cdots & \cdots & & & 0 \\ & \cdots & & & & & & \\ & & 0 & & & & & \\ \vdots & & & d_r & e_r & & & \vdots \\ \vdots & & & e_r & d_{r+1} & & & \\ & & & & & \cdots & & 0 \\ & & & & & & d_{n-2} & e_{n-2} \\ 0 & & & \cdots & & 0 & e_{n-2} & d_{n-1} \end{bmatrix} \tag{11.4.10}$$

Choose k_s equal to the eigenvalue of the leading 2×2 submatrix that is closer to d_r. One can show that the convergence of the algorithm with this strategy is generally cubic (and at worst quadratic for degenerate eigenvalues). This rapid convergence is what makes the algorithm so attractive.

Note that with shifting, the eigenvalues no longer necessarily appear on the diagonal in order of increasing absolute magnitude. The routine `eigsrt` (§11.1) can be used if required.

As we mentioned earlier, the QL decomposition of a general matrix is effected by a sequence of Householder transformations. For a tridiagonal matrix, however, it is more efficient to use plane rotations $\mathbf{P}_{pq}$. One uses the sequence $\mathbf{P}_{01}, \mathbf{P}_{12}, \ldots, \mathbf{P}_{n-2,n-1}$ to annihilate the elements $a_{01}, a_{12}, \ldots, a_{n-2,n-1}$. By symmetry, the subdiagonal elements $a_{10}, a_{21}, \ldots, a_{n-1,n-2}$ will be annihilated too. Thus each $\mathbf{Q}_s$ is a product of plane rotations:

$$\mathbf{Q}_s^T = \mathbf{P}_1^{(s)} \cdot \mathbf{P}_2^{(s)} \cdots \mathbf{P}_{n-1}^{(s)} \tag{11.4.11}$$

where $\mathbf{P}_i$ annihilates $a_{i-1,i}$. Note that it is $\mathbf{Q}^T$ in equation (11.4.11), not $\mathbf{Q}$, because we defined $\mathbf{L} = \mathbf{Q}^T \cdot \mathbf{A}$.

11.4.3 QL Algorithm with Implicit Shifts

The algorithm as described so far can be very successful. However, when the elements of $\mathbf{A}$ differ widely in order of magnitude, subtracting a large k_s from the diagonal elements can lead to loss of accuracy for the small eigenvalues. This difficulty is avoided by the QL algorithm with *implicit shifts*. The implicit QL algorithm is mathematically equivalent to the original QL algorithm, but the computation does not require $k_s\mathbf{1}$ to be actually subtracted from $\mathbf{A}$.

The algorithm is based on the following lemma: If $\mathbf{A}$ is a symmetric nonsingular matrix and $\mathbf{B} = \mathbf{Q}^T \cdot \mathbf{A} \cdot \mathbf{Q}$, where $\mathbf{Q}$ is orthogonal and $\mathbf{B}$ is tridiagonal with positive off-diagonal elements, then $\mathbf{Q}$ and $\mathbf{B}$ are fully determined when the last row of $\mathbf{Q}^T$ is specified. Proof: Let $\mathbf{q}_i^T$ denote the row vector i of the matrix $\mathbf{Q}^T$. Then $\mathbf{q}_i$ is the column vector i of the matrix $\mathbf{Q}$. The relation $\mathbf{B} \cdot \mathbf{Q}^T = \mathbf{Q}^T \cdot \mathbf{A}$ can be written

$$\begin{bmatrix} \beta_0 & \gamma_0 & & & & & \\ \alpha_1 & \beta_1 & \gamma_1 & & & & \\ & & & \vdots & & & \\ & & & \alpha_{n-2} & \beta_{n-2} & \gamma_{n-2} & \\ & & & & \alpha_{n-1} & \beta_{n-1} & \end{bmatrix} \cdot \begin{bmatrix} \mathbf{q}_0^T \\ \mathbf{q}_1^T \\ \vdots \\ \mathbf{q}_{n-2}^T \\ \mathbf{q}_{n-1}^T \end{bmatrix} = \begin{bmatrix} \mathbf{q}_0^T \\ \mathbf{q}_1^T \\ \vdots \\ \mathbf{q}_{n-2}^T \\ \mathbf{q}_{n-1}^T \end{bmatrix} \cdot \mathbf{A} \tag{11.4.12}$$

Row $n-1$ of this matrix equation is

$$\alpha_{n-1}\mathbf{q}_{n-2}^T + \beta_{n-1}\mathbf{q}_{n-1}^T = \mathbf{q}_{n-1}^T \cdot \mathbf{A} \tag{11.4.13}$$

Since $\mathbf{Q}$ is orthogonal,

$$\mathbf{q}_{n-1}^T \cdot \mathbf{q}_m = \delta_{n-1,m} \tag{11.4.14}$$

Thus, if we postmultiply equation (11.4.13) by $\mathbf{q}_{n-1}$, we find

$$\beta_{n-1} = \mathbf{q}_{n-1}^T \cdot \mathbf{A} \cdot \mathbf{q}_{n-1} \tag{11.4.15}$$

which is known since $\mathbf{q}_{n-1}$ is known. Then equation (11.4.13) gives

$$\alpha_{n-1}\mathbf{q}_{n-2}^T = \mathbf{z}_{n-2}^T \tag{11.4.16}$$

where

$$\mathbf{z}_{n-2}^T \equiv \mathbf{q}_{n-1}^T \cdot \mathbf{A} - \beta_{n-1}\mathbf{q}_{n-1}^T \tag{11.4.17}$$

is known. Therefore

$$\alpha_{n-1}^2 = \mathbf{z}_{n-2}^T \mathbf{z}_{n-2}, \tag{11.4.18}$$

or

$$\alpha_{n-1} = |\mathbf{z}_{n-2}| \tag{11.4.19}$$

and

$$\mathbf{q}_{n-2}^T = \mathbf{z}_{n-2}^T / \alpha_{n-1} \tag{11.4.20}$$

(where α_{n-1} is nonzero by hypothesis). Similarly, one can show by induction that if we know $\mathbf{q}_{n-1}, \mathbf{q}_{n-2}, \ldots, \mathbf{q}_{n-j}$ and the α's, β's, and γ's up to level $n-j$, one can determine the quantities at level $n-(j+1)$.

To apply the lemma in practice, suppose one can somehow find a tridiagonal matrix $\bar{\mathbf{A}}_{s+1}$ such that

$$\bar{\mathbf{A}}_{s+1} = \bar{\mathbf{Q}}_s^T \cdot \bar{\mathbf{A}}_s \cdot \bar{\mathbf{Q}}_s \tag{11.4.21}$$

where $\bar{\mathbf{Q}}_s^T$ is orthogonal and has the same last row as $\mathbf{Q}_s^T$ in the original QL algorithm. Then $\bar{\mathbf{Q}}_s = \mathbf{Q}_s$ and $\bar{\mathbf{A}}_{s+1} = \mathbf{A}_{s+1}$.

Now, in the original algorithm, from equation (11.4.11) we see that the last row of $\mathbf{Q}_s^T$ is the same as the last row of $\mathbf{P}_{n-1}^{(s)}$. But recall that $\mathbf{P}_{n-1}^{(s)}$ is a plane rotation designed to annihilate the $(n-2, n-1)$ element of $\mathbf{A}_s - k_s\mathbf{1}$. A simple calculation using the expression (11.1.1) shows that it has parameters

$$c = \frac{d_{n-1} - k_s}{\sqrt{e_{n-1}^2 + (d_{n-1} - k_s)^2}}, \qquad s = \frac{-e_{n-2}}{\sqrt{e_{n-1}^2 + (d_{n-1} - k_s)^2}} \tag{11.4.22}$$

The matrix $\mathbf{P}_{n-1}^{(s)} \cdot \mathbf{A}_s \cdot \mathbf{P}_{n-1}^{(s)T}$ is tridiagonal with two extra elements:

$$\begin{bmatrix} \cdots & & & & & \\ & \times & \times & \times & & \\ & & \times & \times & \times & \mathbf{x} \\ & & & \times & \times & \times \\ & & & \mathbf{x} & \times & \times \end{bmatrix} \tag{11.4.23}$$

We must now reduce this to tridiagonal form with an orthogonal matrix whose last row is $[0, 0, \ldots, 0, 1]$ so that the last row of $\bar{\mathbf{Q}}_s^T$ will stay equal to $\mathbf{P}_{n-1}^{(s)}$. This can be done by a sequence of Householder or Givens transformations. For the special form of the matrix (11.4.23), Givens is better. We rotate in the plane $(n-3, n-2)$ to annihilate the $(n-3, n-1)$ element. [By symmetry, the $(n-1, n-3)$ element will also be zeroed.] This leaves us with a tridiagonal form except for the extra elements $(n-4, n-2)$ and $(n-2, n-4)$. We annihilate these with a rotation in the $(n-4, n-3)$-plane, and so on. Thus a sequence of $n-2$ Givens rotations is required. The result is that

$$\mathbf{Q}_s^T = \bar{\mathbf{Q}}_s^T = \bar{\mathbf{P}}_1^{(s)} \cdot \bar{\mathbf{P}}_2^{(s)} \cdots \bar{\mathbf{P}}_{n-2}^{(s)} \cdot \mathbf{P}_{n-1}^{(s)} \tag{11.4.24}$$

where the $\bar{\mathbf{P}}$'s are the Givens rotations and $\mathbf{P}_{n-1}$ is the same plane rotation as in the original algorithm. Then equation (11.4.21) gives the next iterate of $\mathbf{A}$. Note that the shift k_s enters implicitly through the parameters (11.4.22).

The following routine tqli ("Tridiagonal QL Implicit"), based algorithmically on the implementations in [2,3], works extremely well in practice. The number of iterations for the first few eigenvalues might be four or five, say, but meanwhile the off-diagonal elements in the lower right-hand corner have been reduced too. The later eigenvalues are liberated with very little work. The average number of iterations per eigenvalue is typically 1.3–1.6. The operation count per iteration is $O(n)$, with a fairly large effective coefficient, say $\sim 20n$. The total operation count for the diagonalization is then very roughly $\sim 20n \times (1.3\text{–}1.6)n \sim 30n^2$. If the eigenvectors are required, the statements indicated by comments are included and there is an additional, much larger, workload of about $6n^3$ operations.

eigen_sym.h

```
void Symmeig::tqli()
```

QL algorithm with implicit shifts to determine the eigenvalues and (optionally) the eigenvectors of a real, symmetric, tridiagonal matrix, or of a real symmetric matrix previously reduced by tred2 (§11.3). On input, d[0..n-1] contains the diagonal elements of the tridiagonal matrix. On output, it returns the eigenvalues. The vector e[0..n-1] inputs the subdiagonal elements of the tridiagonal matrix, with e[0] arbitrary. On output e is destroyed. If the eigenvectors of a tridiagonal matrix are desired, the matrix z[0..n-1][0..n-1] is input as the identity matrix. If the eigenvectors of a matrix that has been reduced by tred2 are required, then z is input as the matrix output by tred2. In either case, column k of z returns the normalized eigenvector corresponding to d[k].

```
{
    Int m,l,iter,i,k;
    Doub s,r,p,g,f,dd,c,b;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    for (i=1;i<n;i++) e[i-1]=e[i];                    Convenient to renumber the el-
    e[n-1]=0.0;                                         ements of e.
    for (l=0;l<n;l++) {
        iter=0;
        do {
            for (m=l;m<n-1;m++) {                     Look for a single small subdi-
                dd=abs(d[m])+abs(d[m+1]);               agonal element to split the
                if (abs(e[m]) <= EPS*dd) break;         matrix.
            }
            if (m != l) {
                if (iter++ == 30) throw("Too many iterations in tqli");
                g=(d[l+1]-d[l])/(2.0*e[l]);           Form shift.
                r=pythag(g,1.0);
                g=d[m]-d[l]+e[l]/(g+SIGN(r,g));       This is d_m − k_s.
                s=c=1.0;
                p=0.0;
                for (i=m-1;i>=l;i--) {                A plane rotation as in the origi-
                    f=s*e[i];                           nal QL, followed by Givens
                    b=c*e[i];                           rotations to restore tridiag-
                    e[i+1]=(r=pythag(f,g));             onal form.
                    if (r == 0.0) {                   Recover from underflow.
                        d[i+1] -= p;
                        e[m]=0.0;
                        break;
                    }
                    s=f/r;
                    c=g/r;
                    g=d[i+1]-p;
                    r=(d[i]-g)*s+2.0*c*b;
                    d[i+1]=g+(p=s*r);
                    g=c*r-b;
                    if (yesvecs) {
                        for (k=0;k<n;k++) {           Form eigenvectors.
                            f=z[k][i+1];
                            z[k][i+1]=s*z[k][i]+c*f;
```

```
                        z[k][i]=c*z[k][i]-s*f;
                    }
                }
            }
            if (r == 0.0 && i >= l) continue;
            d[l] -= p;
            e[l]=g;
            e[m]=0.0;
        }
    } while (m != l);
  }
}

Doub Symmeig::pythag(const Doub a, const Doub b) {
```
Computes $(a^2 + b^2)^{1/2}$ without destructive underflow or overflow.
```
    Doub absa=abs(a), absb=abs(b);
    return (absa > absb ? absa*sqrt(1.0+SQR(absb/absa)) :
        (absb == 0.0 ? 0.0 : absb*sqrt(1.0+SQR(absa/absb))));
}
```

11.4.4 Newer Methods

There are two newer algorithms for tridiagonal symmetric systems that are generally more efficient than the QL method, especially for large matrices. The first is the *divide-and-conquer method* [5]. This method divides the tridiagonal matrix into two halves, solves the eigenproblems in each of the two halves, and then stitches the two solutions together to generate the solution of the original problem. The method is applied recursively, with the QL method used once the matrices are sufficiently small. The method is implemented in LAPACK as `dstevd` and is about 2.5 times faster than the QL method for large matrices.

The fastest method of all for the vast majority of matrices is the *MRRR algorithm* (Multiple Relatively Robust Representations) [6]. As we will see in §11.8, inverse iteration can determine the eigenvectors of a tridiagonal matrix in $O(n^2)$ operations. However, clustered eigenvalues lead to eigenvectors that are not properly orthogonal to one another. Using a procedure like Gram-Schmidt to orthogonalize the vectors is $O(n^3)$. The MRRR algorithm is a sophisticated version of inverse iteration that is $O(n^2)$ without requiring Gram-Schmidt. An implementation is available in LAPACK as `dstegr`.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 331–335.[1]

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[3]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.6.4.[4]

Cuppen, J.J.M. 1981, "A Divide-and-Conquer Method for the Symmetric Tridiagonal Eigenproblem," *Numerische Mathematik*, vol. 36, pp. 177–195.[5]

Dhillon, I.S., and Parlett, B.N. 2004, "Multiple Representations to Compute Orthogonal Eigenvectors of Symmetric Tridiagonal Matrices," *Linear Algebra and Its Applications*, vol. 387, pp. 1–28.[6]

11.5 Hermitian Matrices

The complex analog of a real symmetric matrix is a Hermitian matrix, satisfying equation (11.0.4). Jacobi transformations can be used to find eigenvalues and eigenvectors, as can Householder reduction to tridiagonal form followed by QL iteration. Complex versions of the previous routines `jacobi`, `tred2`, and `tqli` are quite analogous to their real counterparts. For working routines, consult [1,2].

An alternative, using the routines in this book, is to convert the Hermitian problem to a real symmetric one: If $\mathbf{C} = \mathbf{A} + i\mathbf{B}$ is a Hermitian matrix, then the $n \times n$ complex eigenvalue problem

$$(\mathbf{A} + i\mathbf{B}) \cdot (\mathbf{u} + i\mathbf{v}) = \lambda(\mathbf{u} + i\mathbf{v}) \tag{11.5.1}$$

is equivalent to the $2n \times 2n$ real problem

$$\begin{bmatrix} \mathbf{A} & -\mathbf{B} \\ \mathbf{B} & \mathbf{A} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \lambda \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} \tag{11.5.2}$$

Note that the $2n \times 2n$ matrix in (11.5.2) is symmetric: $\mathbf{A}^T = \mathbf{A}$ and $\mathbf{B}^T = -\mathbf{B}$ if $\mathbf{C}$ is Hermitian.

Corresponding to a given eigenvalue λ, the vector

$$\begin{bmatrix} -\mathbf{v} \\ \mathbf{u} \end{bmatrix} \tag{11.5.3}$$

is also an eigenvector, as you can verify by writing out the two matrix equations implied by (11.5.2). Thus, if $\lambda_0, \lambda_1, \ldots, \lambda_{n-1}$ are the eigenvalues of $\mathbf{C}$, then the $2n$ eigenvalues of the augmented problem (11.5.2) are $\lambda_0, \lambda_0, \lambda_1, \lambda_1, \ldots, \lambda_{n-1}, \lambda_{n-1}$; each, in other words, is repeated twice. The eigenvectors are pairs of the form $\mathbf{u} + i\mathbf{v}$ and $i(\mathbf{u} + i\mathbf{v})$; that is, they are the same up to an inessential phase. Thus we solve the augmented problem (11.5.2) and choose one eigenvalue and eigenvector from each pair. These give the eigenvalues and eigenvectors of the original matrix $\mathbf{C}$.

Working with the augmented matrix requires a factor of two more storage than the original complex matrix. In principle, a complex algorithm is also a factor of two more efficient in computer time than is the solution of the augmented problem. In practice, most complex implementations do not achieve this factor unless they are written entirely in real arithmetic. (Good library routines always do this.)

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[2]

11.6 Real Nonsymmetric Matrices

The algorithms for symmetric matrices given in the preceding sections are highly satisfactory in practice. By contrast, it is impossible to design equally satisfactory

algorithms for the nonsymmetric case. There are two reasons for this. First, the eigenvalues of a nonsymmetric matrix can be very sensitive to small changes in the matrix elements. Second, the matrix itself can be defective, so that there is no complete set of eigenvectors. We emphasize that these difficulties are intrinsic properties of certain nonsymmetric matrices, and no numerical procedure can "cure" them. The best we can hope for are procedures that don't exacerbate such problems.

The presence of rounding error can only make the situation worse. With finite-precision arithmetic, one cannot even design a foolproof algorithm to determine whether a given matrix is defective or not. Thus current algorithms generally *try* to find a *complete* set of eigenvectors and rely on the user to inspect the results. If any eigenvectors are almost parallel, the matrix is probably defective.

The strategy for finding the eigensystem of a general matrix parallels that of the symmetric case. First we reduce the matrix to a simpler form, and then we perform an iterative procedure on the simplified matrix. The simpler structure we use here is called *Hessenberg* form, defined later in this section. The user interface to the routine is very simple. The declaration

```
Unsymmeig h(a);
```

computes all eigenvalues and eigenvectors of the matrix `a`. The eigenvalues are stored in the complex vector `h.wri` and the corresponding eigenvectors in the columns of the matrix `h.zz`. If `h.wri[i]` is real, the real eigenvector is in `h.zz[0..n-1][i]`. For complex eigenvalues, if `h.wri[i]` has a positive imaginary part, then the complex-conjugate eigenvalue is in `h.wri[i+1]`. Only the eigenvector corresponding to `h.wri[i]` is returned, with the real part in `h.zz[0..n-1][i]` and the imaginary part in `h.zz[0..n-1][i+1]`. The eigenvector corresponding to `h.wri[i+1]` is simply the complex conjugate of this one.

Optional arguments allow you to compute only the eigenvalues, or to input a matrix already in Hessenberg form:

```
Unsymmeig h(a,false);          Only eigenvalues computed.
Unsymmeig h(a,true,true);      Both eigenvalues and eigenvectors, Hessenberg matrix.
```

Here is the routine. The implementations of the various components are discussed in the rest of this section and the next.

eigen_unsym.h

```
struct Unsymmeig {
```

Computes all eigenvalues and eigenvectors of a real nonsymmetric matrix by reduction to Hessenberg form followed by QR iteration.

```
    Int n;
    MatDoub a,zz;
    VecComplex wri;
    VecDoub scale;                  Stores scaling from balance.
    VecInt perm;                    Stores permutation from elmhes.
    Bool yesvecs,hessen;

    Unsymmeig(MatDoub_I &aa, Bool yesvec=true, Bool hessenb=false) :
        n(aa.nrows()), a(aa), zz(n,n,0.0), wri(n), scale(n,1.0), perm(n),
        yesvecs(yesvec), hessen(hessenb)
```

Computes all eigenvalues and (optionally) eigenvectors of a real nonsymmetric matrix `a[0..n-1][0..n-1]` by reduction to Hessenberg form followed by QR iteration. If `yesvecs` is input as `true` (the default), then the eigenvectors are computed. Otherwise, only the eigenvalues are computed. If `hessen` is input as `false` (the default), the matrix is first reduced to Hessenberg form. Otherwise it is assumed that the matrix is already in Hessenberg from. On output, `wri[0..n-1]` contains the eigenvalues of a sorted into descending order, while `zz[0..n-1][0..n-1]` is a matrix whose columns contain the corresponding

```
    eigenvectors. For a complex eigenvalue, only the eigenvector corresponding to the eigen-
    value with a positive imaginary part is stored, with the real part in zz[0..n-1][i] and the
    imaginary part in h.zz[0..n-1][i+1]. The eigenvectors are not normalized.
    {
        balance();
        if (!hessen) elmhes();
        if (yesvecs) {
            for (Int i=0;i<n;i++)          Initialize to unit matrix.
                zz[i][i]=1.0;
            if (!hessen) eltran();
            hqr2();
            balbak();
            sortvecs();
        } else {
            hqr();
            sort();
        }
    }
    void balance();
    void elmhes();
    void eltran();
    void hqr();
    void hqr2();
    void balbak();
    void sort();
    void sortvecs();
};
```

11.6.1 Balancing

The sensitivity of eigenvalues to rounding errors during the execution of some algorithms can be reduced by the procedure of *balancing*. The errors in the eigensystem found by a numerical procedure are generally proportional to the Euclidean norm of the matrix, that is, to the square root of the sum of the squares of the elements. The idea of balancing is to use similarity transformations to make corresponding rows and columns of the matrix have comparable norms, thus reducing the overall norm of the matrix while leaving the eigenvalues unchanged. A symmetric matrix is already balanced.

Balancing is a procedure with of order N^2 operations. Thus, the time taken by the procedure `balance`, given below, should never be significant compared to the total time required to find the eigenvalues. It is therefore recommended that you *always* balance nonsymmetric matrices. It never hurts, and it can substantially improve the accuracy of the eigenvalues computed for a badly balanced matrix.

The actual algorithm used is due to Osborne, as discussed in [1]. It consists of a sequence of similarity transformations by diagonal matrices $\mathbf{D}$. To avoid introducing rounding errors during the balancing process, the elements of $\mathbf{D}$ are restricted to be exact powers of the radix base employed for floating-point arithmetic (i.e., 2 for all modern machines, but 16 for some historical mainframe architectures). The output is a matrix that is balanced in the norm given by summing the absolute magnitudes of the matrix elements. This is more efficient than using the Euclidean norm, and equally effective: A large reduction in one norm implies a large reduction in the other.

Note that if the off-diagonal elements of any row or column of a matrix are all zero, then the diagonal element is an eigenvalue. If the eigenvalue happens to be ill-conditioned (sensitive to small changes in the matrix elements), it will have

relatively large errors when determined by the routine hqr (§11.7). Had we merely inspected the matrix beforehand, we could have determined the isolated eigenvalue exactly and then deleted the corresponding row and column from the matrix. You should consider whether such a pre-inspection might be useful in your application. (For symmetric matrices, the routines we gave will determine isolated eigenvalues accurately in all cases.)

The routine balance keeps track of the scale factors used in the balancing. If you are computing eigenvectors as well as eigenvalues, then the accumulated similarity transformation of the original matrix is undone by applying these scale factors in the routine balbak.

```
void Unsymmeig::balance()                                              eigen_unsym.h
```
Given a matrix a[0..n-1][0..n-1], this routine replaces it by a balanced matrix with identical eigenvalues. A symmetric matrix is already balanced and is unaffected by this procedure.

```
{
    const Doub RADIX = numeric_limits<Doub>::radix;
    Bool done=false;
    Doub sqrdx=RADIX*RADIX;
    while (!done) {
        done=true;
        for (Int i=0;i<n;i++) {                    Calculate row and column norms.
            Doub r=0.0,c=0.0;
            for (Int j=0;j<n;j++)
                if (j != i) {
                    c += abs(a[j][i]);
                    r += abs(a[i][j]);
                }
            if (c != 0.0 && r != 0.0) {            If both are nonzero,
                Doub g=r/RADIX;
                Doub f=1.0;
                Doub s=c+r;
                while (c<g) {                      find the integer power of the machine
                    f *= RADIX;                        radix that comes closest to balanc-
                    c *= sqrdx;                        ing the matrix.
                }
                g=r*RADIX;
                while (c>g) {
                    f /= RADIX;
                    c /= sqrdx;
                }
                if ((c+r)/f < 0.95*s) {
                    done=false;
                    g=1.0/f;
                    scale[i] *= f;
                    for (Int j=0;j<n;j++) a[i][j] *= g;   Apply similarity transforma-
                    for (Int j=0;j<n;j++) a[j][i] *= f;       tion.
                }
            }
        }
    }
}
```

```
void Unsymmeig::balbak()                                               eigen_unsym.h
```
Forms the eigenvectors of a real nonsymmetric matrix by back transforming those of the corresponding balanced matrix determined by balance.

```
{
    for (Int i=0;i<n;i++)
        for (Int j=0;j<n;j++)
            zz[i][j] *= scale[i];
}
```

11.6.2 Reduction to Hessenberg Form

An *upper Hessenberg* matrix has zeros everywhere below the diagonal except for the first subdiagonal row. For example, in the 6×6 case, the nonzero elements are

$$\begin{bmatrix} \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \\ & \times & \times & \times & \times & \times \\ & & \times & \times & \times & \times \\ & & & \times & \times & \times \\ & & & & \times & \times \end{bmatrix}$$

By now you should be able to tell at a glance that such a structure can be achieved by a sequence of Householder transformations, each one zeroing the required elements in a column of the matrix. Householder reduction to Hessenberg form is in fact an accepted technique. An alternative, however, is a procedure analogous to Gaussian elimination with pivoting. We will use this elimination procedure since it is about a factor of two more efficient than the Householder method, and also since we want to teach you the method. It is possible to construct matrices for which the Householder reduction, being orthogonal, is stable and elimination is not, but such matrices are extremely rare in practice.

Straight Gaussian elimination is not a similarity transformation of the matrix. Accordingly, the actual elimination procedure used is slightly different. We proceed in a series of stages $r = 1, 2, \ldots, N-2$. Before the rth stage, the original matrix $\mathbf{A} \equiv \mathbf{A}_1$ has become $\mathbf{A}_r$, which is upper Hessenberg up to, but not including, row and column $r-1$. The rth stage then consists of the following sequence of operations:

- Find the element of maximum magnitude in column $r-1$ below the diagonal. If it is zero, skip the next two "bullets" and the stage is done. Otherwise, suppose the maximum element was in row r'.
- Interchange rows r' and r. This is the pivoting procedure. To make the permutation a similarity transformation, also interchange columns r' and r.
- For $i = r+1, r+2, \ldots, N-1$, compute the multiplier

$$n_{ir} \equiv \frac{a_{i,r-1}}{a_{r,r-1}}$$

  Subtract n_{ir} times row r from row i. To make the elimination a similarity transformation, also *add* n_{ir} times column i to column r.

A total of $N-2$ such stages are required.

When the magnitudes of the matrix elements vary over many orders, you should try to rearrange the matrix so that the largest elements are in the top left-hand corner. This reduces the roundoff error, since the reduction proceeds from left to right.

The routine `elmhes` keeps track of the permutations applied during the elimination. If you are computing eigenvectors, then the accumulated similarity transformation is applied to the eigenvectors by the routine `eltran`, which includes any necessary permutations. The operation count is about $5N^3/3$ for large N.

`void Unsymmeig::elmhes()` eigen_unsym.h

Reduction to Hessenberg form by the elimination method. Replaces the real nonsymmetric matrix `a[0..n-1][0..n-1]` by an upper Hessenberg matrix with identical eigenvalues. Recommended, but not required, is that this routine be preceded by `balance`. On output, the Hessenberg matrix is in elements `a[i][j]` with `i` $\leq$ `j+1`. Elements with `i` $>$ `j+1` are to be thought of as zero, but are returned with random values.

```
{
    for (Int m=1;m<n-1;m++) {           m is called r in the text.
        Doub x=0.0;
        Int i=m;
        for (Int j=m;j<n;j++) {         Find the pivot.
            if (abs(a[j][m-1]) > abs(x)) {
                x=a[j][m-1];
                i=j;
            }
        }
        perm[m]=i;                      Store permutation.
        if (i != m) {                   Interchange rows and columns.
            for (Int j=m-1;j<n;j++) SWAP(a[i][j],a[m][j]);
            for (Int j=0;j<n;j++) SWAP(a[j][i],a[j][m]);
        }
        if (x != 0.0) {                 Carry out the elimination.
            for (i=m+1;i<n;i++) {
                Doub y=a[i][m-1];
                if (y != 0.0) {
                    y /= x;
                    a[i][m-1]=y;
                    for (Int j=m;j<n;j++) a[i][j] -= y*a[m][j];
                    for (Int j=0;j<n;j++) a[j][m] += y*a[j][i];
                }
            }
        }
    }
}
```

`void Unsymmeig::eltran()` eigen_unsym.h

This routine accumulates the stabilized elementary similarity transformations used in the reduction to upper Hessenberg form by `elmhes`. The multipliers that were used in the reduction are obtained from the lower triangle (below the subdiagonal) of `a`. The transformations are permuted according to the permutations stored in `perm` by `elmhes`.

```
{
    for (Int mp=n-2;mp>0;mp--) {
        for (Int k=mp+1;k<n;k++)
            zz[k][mp]=a[k][mp-1];
        Int i=perm[mp];
        if (i != mp) {
            for (Int j=mp;j<n;j++) {
                zz[mp][j]=zz[i][j];
                zz[i][j]=0.0;
            }
            zz[i][mp]=1.0;
        }
    }
}
```

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[2]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.5.4.[3]

11.7 The QR Algorithm for Real Hessenberg Matrices

To complete the strategy for real, nonsymmetric matrices that was laid out in §11.6, we need to compute the eigenvalues and eigenvectors of a real Hessenberg matrix. Recall the following relations for the QR algorithm with shifts:

$$\mathbf{Q}_s \cdot (\mathbf{A}_s - k_s \mathbf{1}) = \mathbf{R}_s \tag{11.7.1}$$

where $\mathbf{Q}$ is orthogonal and $\mathbf{R}$ is upper triangular, and

$$\begin{aligned} \mathbf{A}_{s+1} &= \mathbf{R}_s \cdot \mathbf{Q}_s^T + k_s \mathbf{1} \\ &= \mathbf{Q}_s \cdot \mathbf{A}_s \cdot \mathbf{Q}_s^T \end{aligned} \tag{11.7.2}$$

The QR transformation preserves the upper Hessenberg form of the original matrix $\mathbf{A} \equiv \mathbf{A}_1$, and the workload on such a matrix is $O(n^2)$ per iteration as opposed to $O(n^3)$ on a general matrix. As $s \to \infty$, $\mathbf{A}_s$ converges to a form where the eigenvalues are either isolated on the diagonal or are eigenvalues of a 2×2 submatrix on the diagonal.

As we pointed out in §11.4, shifting is essential for rapid convergence. A key difference here is that a nonsymmetric real matrix can have complex eigenvalues. This means that good choices for the shifts k_s may be complex, apparently necessitating complex arithmetic.

Complex arithmetic can be avoided, however, by a clever trick. This trick, plus a detailed description of how the QR algorithm is used, is described in a Webnote [1].

The operation count for the QR algorithm for Hessenberg matrices is $\sim 10k^2$ per iteration, where k is the current size of the matrix. The typical average number of iterations per eigenvalue is about two, so the total operation count for all the eigenvalues is $\sim 10n^3$. The total operation count for both eigenvalues and eigenvectors is $\sim 25n^3$.

The routines `hqr` for the eigenvalues only, and `hqr2`, which computes both eigenvalues and eigenvectors, are given in full in a Webnote [2], along with a few `Unsymmeig` utility routines not already listed. The implementations are based algorithmically on the above description, in turn following the implementations in [3,4].

CITED REFERENCES AND FURTHER READING:

Numerical Recipes Software 2007, "Description of the QR Algorithm for Hessenberg Matrices," *Numerical Recipes Webnote No. 16*, at `http://www.nr.com/webnotes?16` [1]

Numerical Recipes Software 2007, "Implementations in Unsymmeig," *Numerical Recipes Webnote No. 17*, at `http://www.nr.com/webnotes?17` [2]

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[3]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §7.5.

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[4]

11.8 Improving Eigenvalues and/or Finding Eigenvectors by Inverse Iteration

The basic idea behind inverse iteration is quite simple. Let $\mathbf{y}$ be the solution of the linear system

$$(\mathbf{A} - \tau\mathbf{1}) \cdot \mathbf{y} = \mathbf{b} \tag{11.8.1}$$

where $\mathbf{b}$ is a random vector and τ is close to some eigenvalue λ of $\mathbf{A}$. Then the solution $\mathbf{y}$ will be close to the eigenvector corresponding to λ. The procedure can be iterated: Replace $\mathbf{b}$ by $\mathbf{y}$ and solve for a new $\mathbf{y}$, which will be even closer to the true eigenvector.

We can see why this works by expanding both $\mathbf{y}$ and $\mathbf{b}$ as linear combinations of the eigenvectors $\mathbf{x}_j$ of $\mathbf{A}$:

$$\mathbf{y} = \sum_j \alpha_j \mathbf{x}_j \qquad \mathbf{b} = \sum_j \beta_j \mathbf{x}_j \tag{11.8.2}$$

Then (11.8.1) gives

$$\sum_j \alpha_j (\lambda_j - \tau)\mathbf{x}_j = \sum_j \beta_j \mathbf{x}_j \tag{11.8.3}$$

so that

$$\alpha_j = \frac{\beta_j}{\lambda_j - \tau} \tag{11.8.4}$$

and

$$\mathbf{y} = \sum_j \frac{\beta_j \mathbf{x}_j}{\lambda_j - \tau} \tag{11.8.5}$$

If τ is close to λ_n, say, then provided β_n is not accidentally too small, $\mathbf{y}$ will be approximately $\mathbf{x}_n$, up to a normalization. Moreover, each iteration of this procedure gives another power of $\lambda_j - \tau$ in the denominator of (11.8.5). Thus the convergence is rapid for well-separated eigenvalues.

Suppose at the kth stage of iteration we are solving the equation

$$(\mathbf{A} - \tau_k\mathbf{1}) \cdot \mathbf{y} = \mathbf{b}_k \tag{11.8.6}$$

where $\mathbf{b}_k$ and τ_k are our current guesses for some eigenvector and eigenvalue of interest (let's say $\mathbf{x}_n$ and λ_n). Normalize $\mathbf{b}_k$ so that $\mathbf{b}_k \cdot \mathbf{b}_k = 1$. The exact eigenvector and eigenvalue satisfy

$$\mathbf{A} \cdot \mathbf{x}_n = \lambda_n \mathbf{x}_n \tag{11.8.7}$$

so

$$(\mathbf{A} - \tau_k\mathbf{1}) \cdot \mathbf{x}_n = (\lambda_n - \tau_k)\mathbf{x}_n \tag{11.8.8}$$

Since $\mathbf{y}$ of (11.8.6) is an improved approximation to $\mathbf{x}_n$, we normalize it and set

$$\mathbf{b}_{k+1} = \frac{\mathbf{y}}{|\mathbf{y}|} \tag{11.8.9}$$

We get an improved estimate of the eigenvalue by substituting our improved guess $\mathbf{y}$ for $\mathbf{x}_n$ in (11.8.8). By (11.8.6), the left-hand side is $\mathbf{b}_k$, so, calling λ_n our new value τ_{k+1}, we find

$$\tau_{k+1} = \tau_k + \frac{1}{\mathbf{b}_k \cdot \mathbf{y}} \tag{11.8.10}$$

While the above formulas look simple enough, in practice the implementation can be quite tricky. The first question to be resolved is *when* to use inverse iteration. Most of the computational load occurs in solving the linear system (11.8.6). Thus a possible strategy is first to reduce the matrix $\mathbf{A}$ to a special form that allows easy solution of (11.8.6). Tridiagonal form for symmetric matrices or Hessenberg for nonsymmetric are the obvious choices. The tridiagonal form can be solved in $O(N)$ operations, the Hessenberg form in $O(N^2)$ operations. If you then apply inverse iteration to generate all the eigenvectors, this gives an $O(N^2)$ method for tridiagonal matrices. The problem is that closely spaced eigenvalues lead to eigenvectors that are not properly orthogonal to one another. Using a procedure like Gram-Schmidt to orthogonalize the vectors is $O(n^3)$, and not entirely satisfactory anyway. Accordingly, inverse iteration is generally used when one already has good eigenvalues and wants only a few selected eigenvectors.

You can write a simple inverse iteration routine yourself using LU decomposition to solve (11.8.6). You can decide whether to use the general LU algorithm we gave in Chapter 2 or whether to take advantage of tridiagonal or Hessenberg form. Note that, since the linear system (11.8.6) is nearly singular, you must be careful to use a version of LU decomposition like that in §2.3 which replaces a zero pivot with a very small number.

We have chosen not to give a general inverse iteration routine in this book, because it is quite cumbersome to take account of all the cases that can arise. Routines are given, for example, in [1-3]. If you use these, or write your own routine, you may appreciate the following pointers.

One starts by supplying an initial value τ_0 for the eigenvalue λ_n of interest. Choose a random normalized vector $\mathbf{b}_0$ as the initial guess for the eigenvector $\mathbf{x}_n$, and solve (11.8.6). The new vector $\mathbf{y}$ is bigger than $\mathbf{b}_0$ by a "growth factor" $|\mathbf{y}|$, which ideally should be large. Equivalently, the change in the eigenvalue, which by (11.8.10) is essentially $1/|\mathbf{y}|$, should be small. The following cases can arise:

- If the growth factor is too small initially, then we assume we have made a "bad" choice of random vector. This can happen not just because of a small β_n in (11.8.5), but also in the case of a defective matrix, when (11.8.5) does not even apply (see, e.g., [1] or [4] for details). We go back to the beginning and choose a new initial vector.
- The change $|\mathbf{b}_1 - \mathbf{b}_0|$ might be less than some tolerance ϵ. We can use this as a criterion for stopping, iterating until it is satisfied, with a maximum of 5–10 iterations, say.
- After a few iterations, if $|\mathbf{b}_{k+1} - \mathbf{b}_k|$ is not decreasing rapidly enough, we can try updating the eigenvalue according to (11.8.10). If $\tau_{k+1} = \tau_k$ to machine accuracy, we are not going to improve the eigenvector much more and can quit. Otherwise start another cycle of iterations with the new eigenvalue.

The reason we do not update the eigenvalue at every step is that when we solve the linear system (11.8.6) by LU decomposition, we can save the decomposition if

τ_k is fixed (assuming we are working with the full matrix). We only need to do the backsubstitution step each time we update $\mathbf{b}_k$. The number of iterations we decide to do with a fixed τ_k is a trade-off between the quadratic convergence but $O(N^3)$ workload for updating τ_k at each step and the linear convergence but $O(N^2)$ load for keeping τ_k fixed. If you have determined the eigenvalue by one of the routines given earlier in the chapter, it is probably correct to machine accuracy anyway, and you can omit updating it.

There are two different pathologies that can arise during inverse iteration. The first is multiple or closely spaced roots. This is more often a problem with symmetric matrices. Inverse iteration will find only one eigenvector for a given initial guess τ_0. A good strategy is to perturb the last few significant digits in τ_0 and then repeat the iteration. Usually this provides an independent eigenvector. Special steps generally have to be taken to ensure orthogonality of the linearly independent eigenvectors, whereas the Jacobi and QL algorithms automatically yield orthogonal eigenvectors even in the case of multiple eigenvalues.

The second problem, peculiar to nonsymmetric matrices, is the defective case. Unless one makes a "good" initial guess, the growth factor is small. Moreover, iteration does not improve matters. In this case, the remedy is to choose random initial vectors, solve (11.8.6) once, and quit as soon as *any* vector gives an acceptably large growth factor. Typically only a few trials are necessary.

One further complication in the nonsymmetric case is that a real matrix can have complex-conjugate pairs of eigenvalues. You will then have to use complex arithmetic to solve (11.8.6) for the complex eigenvectors. For any moderate-sized (or larger) nonsymmetric matrix, our recommendation is to avoid inverse iteration in favor of a QR method like `Unsymmeig`.

A good discussion of these and other problems with inverse iteration is given in [5]. As discussed in §11.4.4, for symmetric tridiagonal matrices, the MRRR algorithm is a sophisticated version of inverse iteration that avoids all these problems.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America).

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), p. 418.[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[2]

Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, `http://www.netlib.org/lapack`.[3]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.6.3.[4]

Dhillon, I.S. 1998, "Current Inverse Iteration Software Can Fail," *BIT Numerical Mathematics*, vol. 38, pp. 685–704.[5]